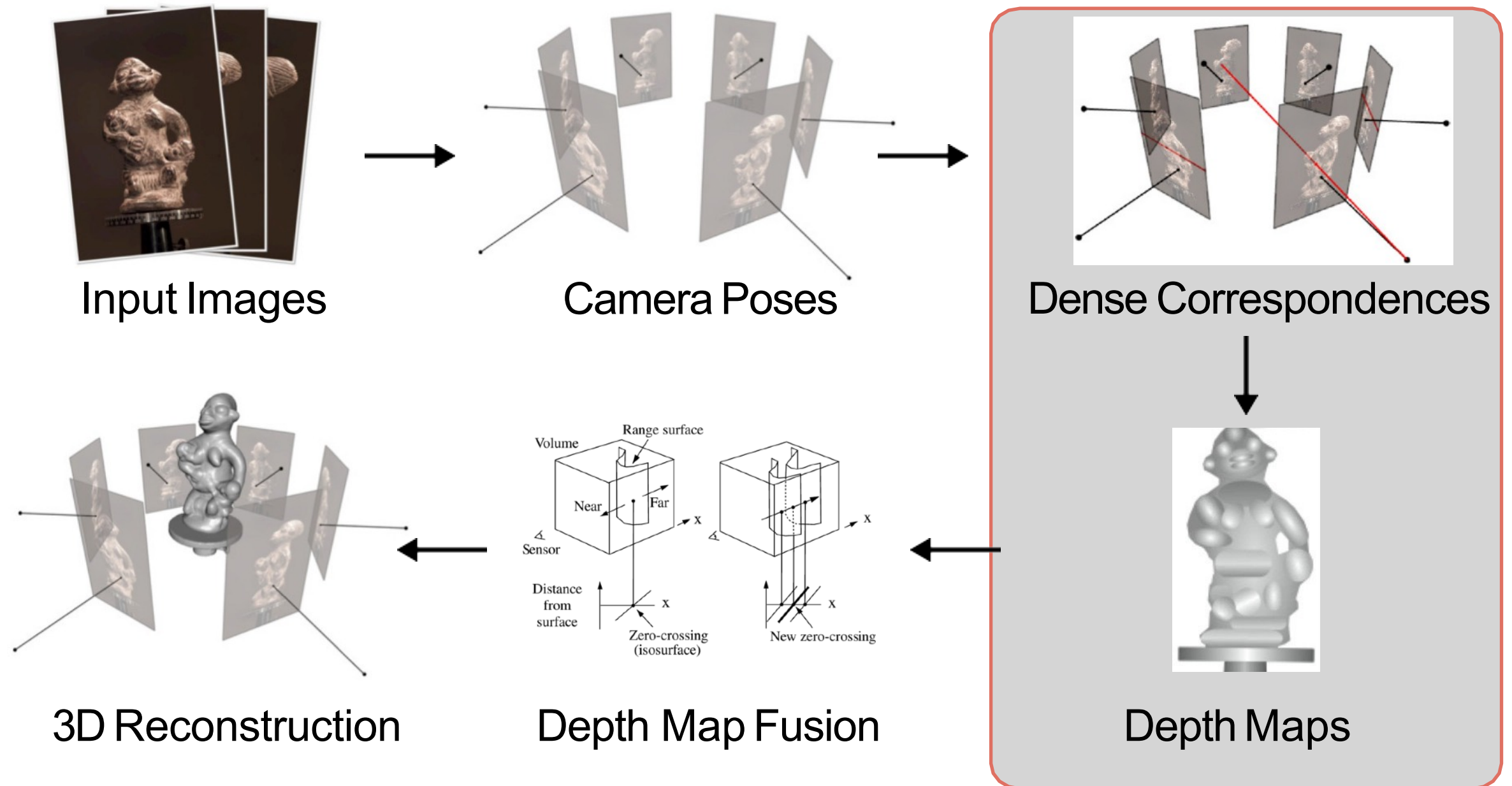


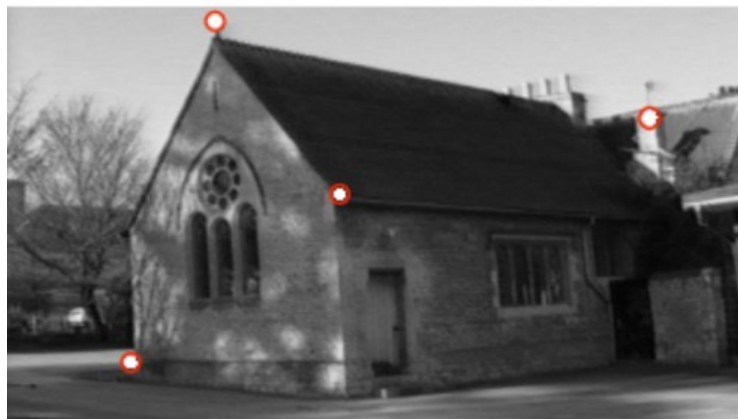
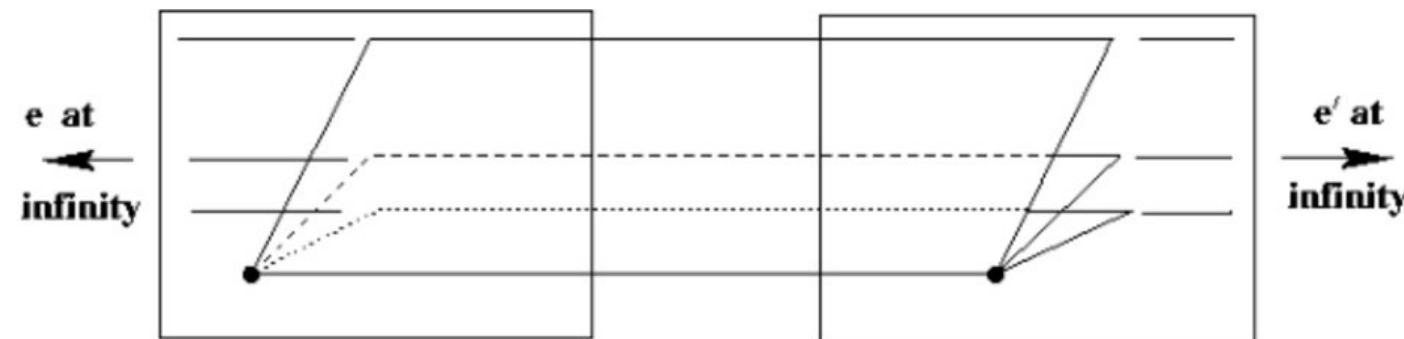
3D Reconstruction Pipeline



Last lectures: Stereo reconstruction

Rectification

What if both cameras face exactly the same direction?



- Image planes are co-planar $\Rightarrow$ Epipoles at infinity, epipolar lines parallel
- Correspondences search along horizontal scanlines (simplifies implementation)

Disparity estimation



- The relative horizontal displacement is called **disparity** (here: 0-160 px to the left)
- Disparity is anti-proportional to depth (far points move less than closer points)

Disparity estimation



- The relative horizontal displacement is called **disparity** (here: 0-160 px to the left)
- Disparity is anti-proportional to depth (far points move less than closer points)

Disparity to depth

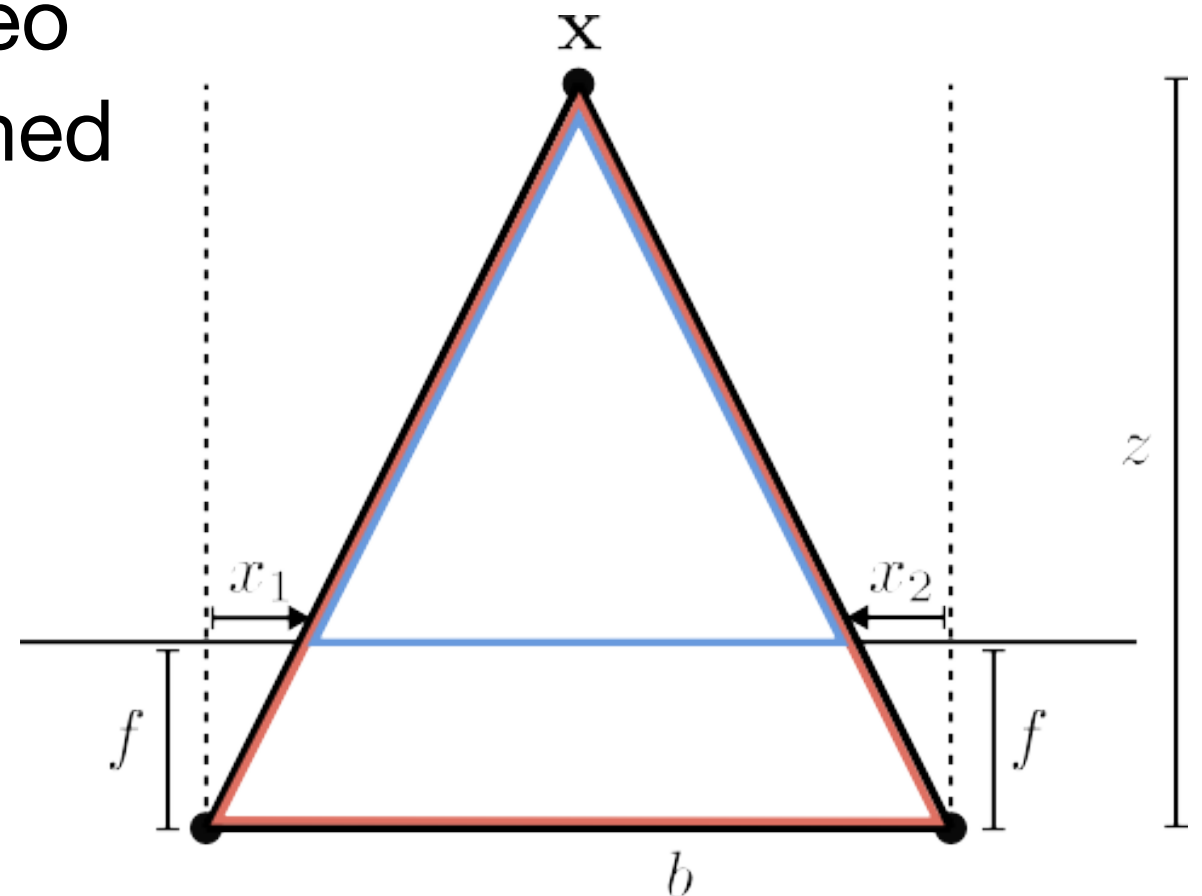
The left and right ray intersect
(shared epipolar plane).

Assume the disparity is $d = x_1 - x_2$.
Given a focal length f and the stereo
baseline b , the depth z is constrained
by

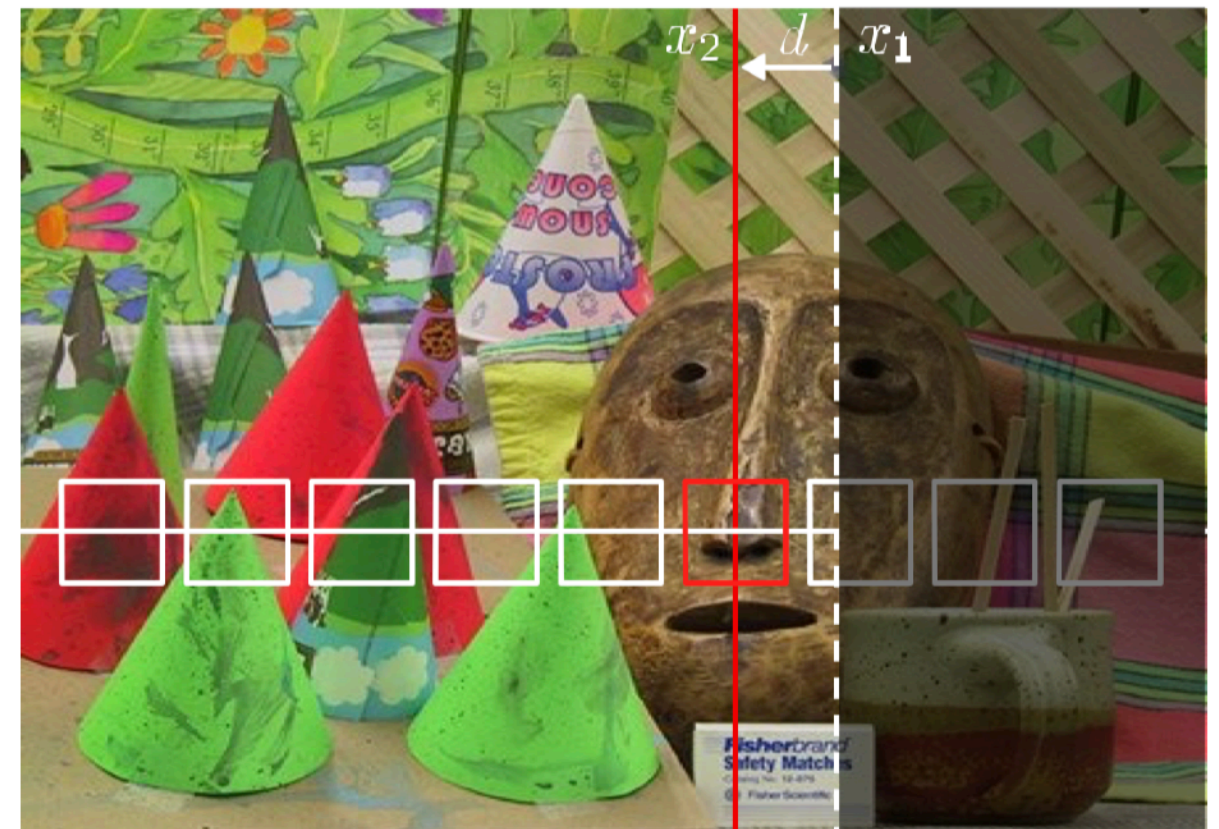
$$\frac{z - f}{b - d} = \frac{z}{b}$$

$$zb - fb = zb - zd$$

$$z = \frac{fb}{d}$$

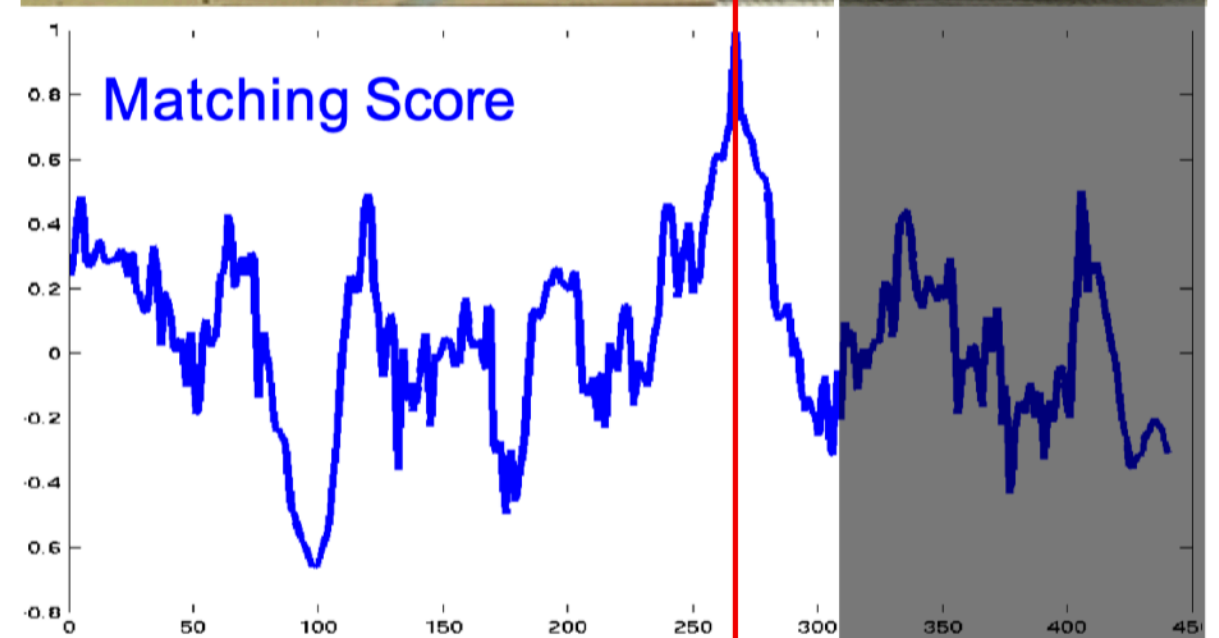


Stereo matching

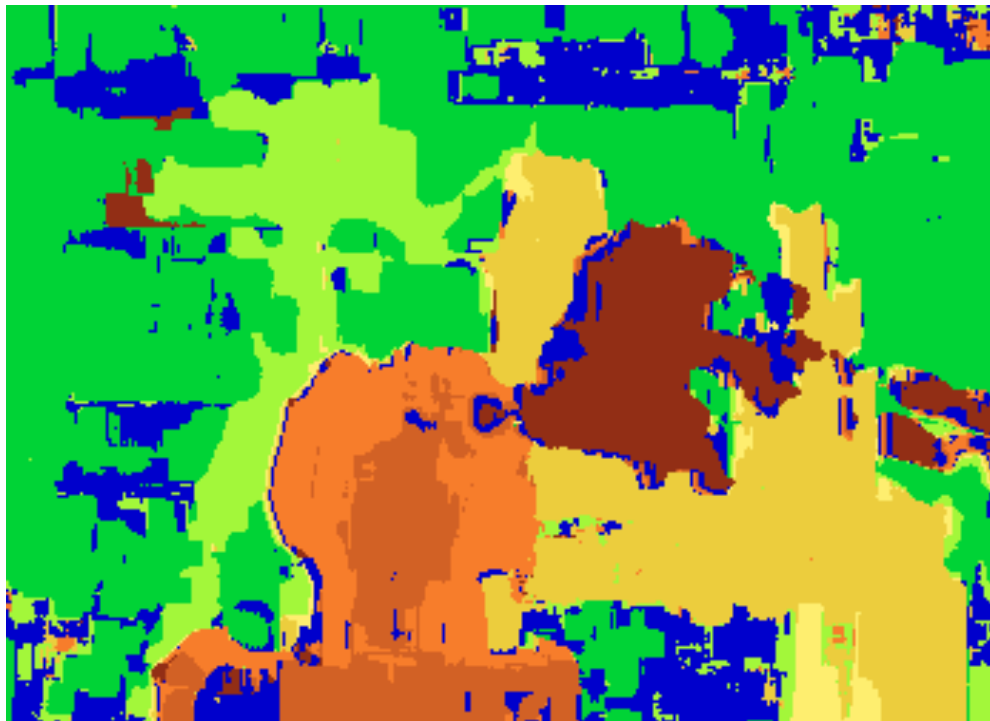


Block matching

- Choose disparity range
- For all pixels, compute best disparity
- Repeat in opposite direction, remove inconsistencies to reduce outliers



Block matching result



Window-based matching
(best window size)



Ground truth



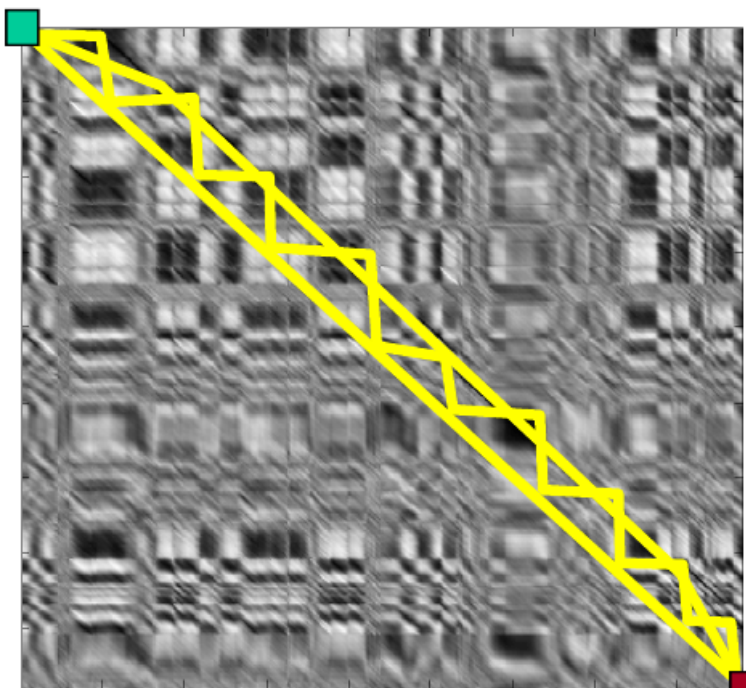
Despite all these problems: The „real“ depth is in practice mostly smooth - except at typically rare discontinuities such as object boundaries.

Basic idea

- We want to find the correct disparity at each pixel by matching it to the corresponding point in the other image
 - We match image patches centered at the respective pixels
 - From disparity, we obtain depth
- The match should optimize similarity of the patches
 - this is an independent objective at each pixel
- At the same time, matches of neighboring pixels should usually lead to similar depth estimates
 - this objective refers to results of pairs of neighboring pixels

Cox et.al. Stereo Matching

Start



End

Recap: want to find lowest cost path from upper left to lower right of DSI image.

At each point on the path we have three choices: step left, step down, step diagonally.

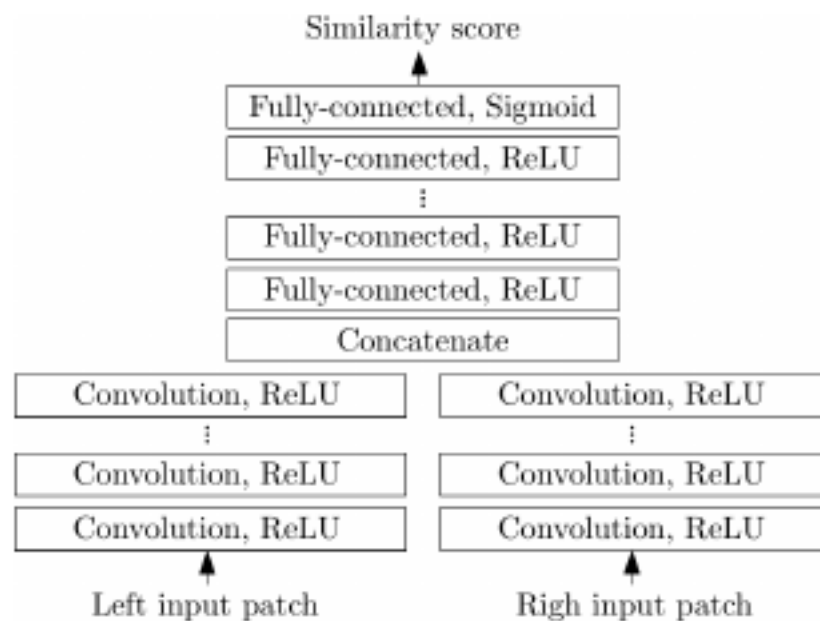
Each choice has a well-defined cost associated with it.

This problem just screams out for Dynamic Programming!
(which, indeed, is how Cox et.al. solve the problem)

Siamese Network Architecture

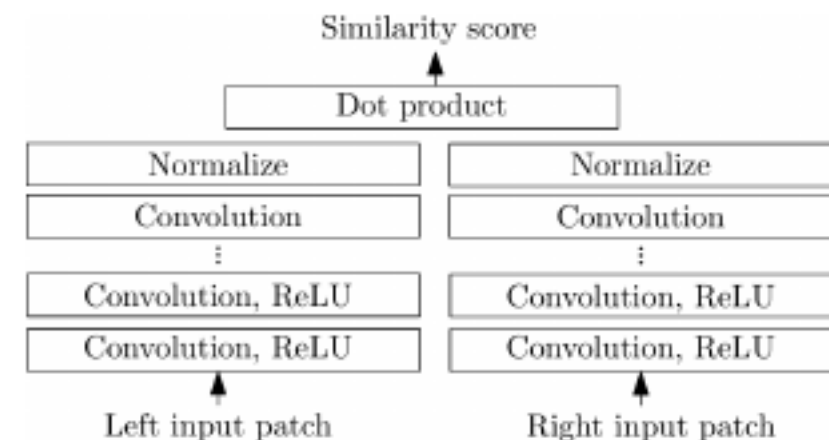
Learned Similarity:

- ▶ Learn features & sim. metric
- ▶ Potentially more expressive
- ▶ Slow (WxHxD MLP evaluations)



Cosine Similarity:

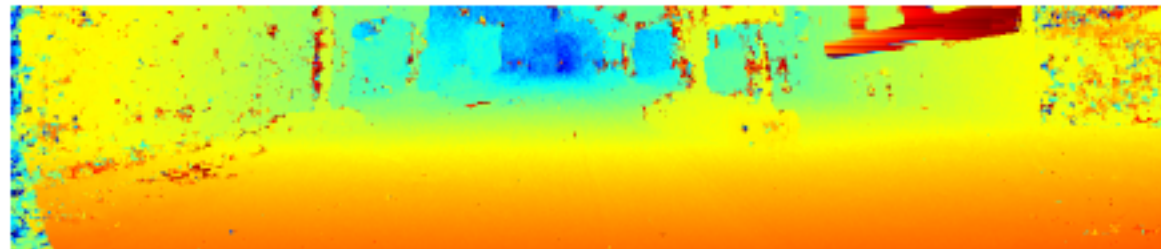
- ▶ Learn features & apply dot-product
- ▶ Features must do the heavy lifting
- ▶ Fast matching (no network eval.)



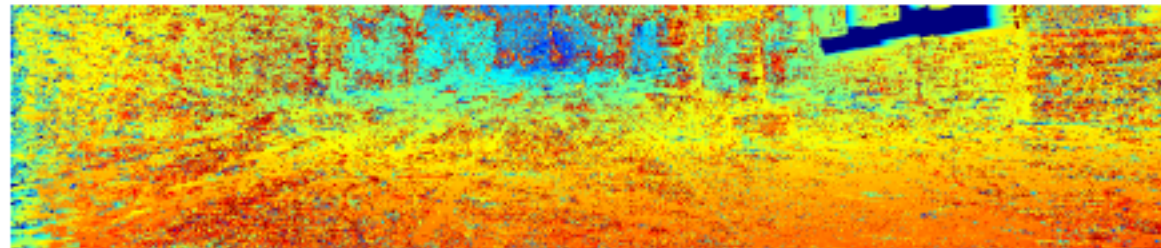
Winner-takes-All Results



Left Input Image



Siamese Network

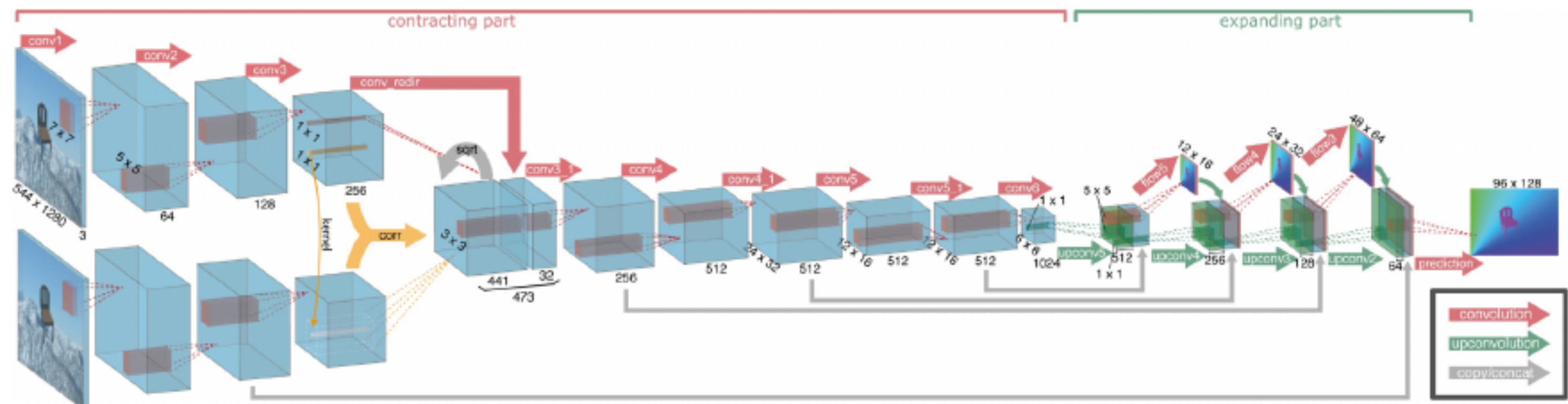


Standard Block Matching

Zbontar and LeCun: Stereo Matching by Training a Convolutional Neural Network to Compare Image Patches. JMLR, 2016.

45

DispNet

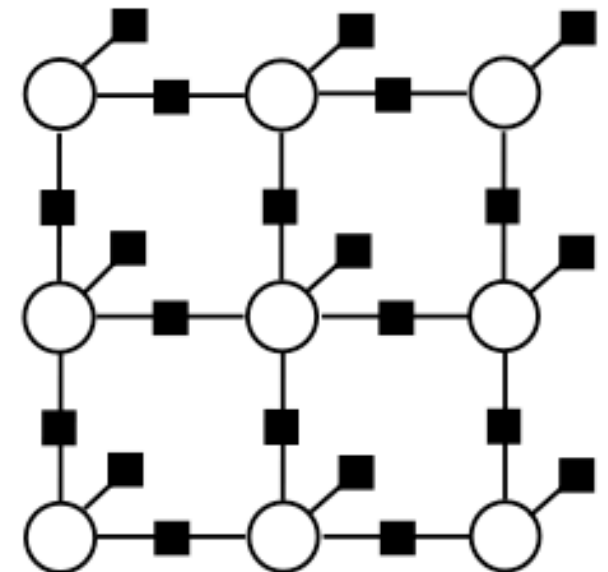


- ▶ DispNet was the first end-to-end trained deep neural network for stereo
- ▶ It used a U-Net like architecture with skip-Connections to retain details
- ▶ Correlation layer (40px displacement corresponding to 160px in input image)
- ▶ Multi-scale loss (disparity error in pixels), curriculum learning (learn easy-to-hard)

A graphical model (of order 2)

- Dependencies of parameters are modeled as a graph
 - here corresponding to 4-neighborhoods on the pixel grid
- We find a solution minimizing both
 - the **data cost** at each node (=pixel)
 - the **smoothness cost** at each edge (=neighboring pixel pair)
- This is modeled as an exponential probability distribution over unary and binary terms connected to the graph

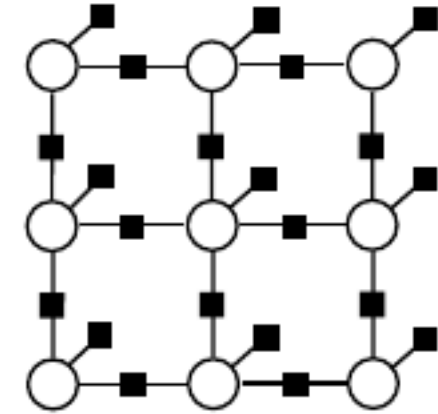
$$p(\mathbf{D}) \propto \exp \left\{ - \sum_i \psi_{data}(d_i) - \lambda \sum_{i \sim j} \psi_{smooth}(d_i, d_j) \right\}$$



Dense Stereo MRF

Markov Random Field (MRF) from last slide:

$$p(\mathbf{D}) \propto \exp \left\{ - \underbrace{\left(\sum_i \psi_{data}(d_i) + \lambda \sum_{i \sim j} \psi_{smooth}(d_i, d_j) \right)}_{\text{Energy } E} \right\}$$



- ▶ $i \sim j$: neighboring pixels on a 4-connected grid
- ▶ **Unary terms:** Matching cost $\psi_{data}(d)$
- ▶ **Pairwise terms:** Encourages smoothness between adjacent pixels, e.g.:
 - ▶ Potts model: $\psi_{smooth}(d, d') = [d \neq d']$
 - ▶ Truncated l_1 penalty: $\psi_{smooth}(d, d') = \min(|d - d'|, \tau)$
- ▶ The parameter λ is added to control the strength of the smoothness prior
- ▶ Solve MRF approximately for disparity map $\mathbf{D}$ using BP, graph cuts, etc.

MRF result (via graph cuts)



Graph cuts-based method



Ground truth

Boykov et al., [Fast Approximate Energy Minimization via Graph Cuts](#),

International Conference on Computer Vision 1999.

Suggested reading

A Comparative Study of Modern Inference Techniques for Structured Discrete Energy Minimization Problems

Jörg H. Kappes Bjoern Andres Fred A. Hamprecht Christoph Schnörr Sebastian Nowozin
Dhruv Batra Sungwoong Kim Bernhard X. Kausler Thorben Kröger Jan Lellmann
Nikos Komodakis Bogdan Savchynskyy Carsten Rother

April 1, 2014

arXiv:1404.0533v1 [cs.CV] 2 Apr 2014

Abstract

Szeliski et al. published an influential study in 2006 on energy minimization methods for Markov Random Fields (MRF). This study provided valuable insights in choosing the best optimization technique for certain classes of problems.

While these insights remain generally useful today, the phenomenal success of random field models means that the *kinds* of inference problems that have to be solved changed significantly. Specifically, the models today often include higher order interactions, flexible connectivity structures, large label-spaces of different cardinalities, or learned energy tables. To reflect these changes, we provide a modernized and enlarged study. We present an empirical comparison of 32 state-of-the-art optimization techniques on a corpus of 2,453 energy minimization instances from diverse applications in computer vision. To ensure reproducibility, we evaluate all methods in the OpenGM 2 framework and report extensive results regarding runtime and solution quality. Key insights from our study agree with the results of Szeliski et al. for the types of models they studied. However, on new and challenging types of models our findings disagree and suggest that polyhedral methods and integer programming solvers are competitive in terms of runtime and solution quality over a large range of model types.

1 Introduction

Discrete energy minimization problems, in the form of factor graphs, or equivalently Markov or Conditional Random Field models (MRF/CRF) are a mainstay of computer vision research. Their applications are diverse and range from image denoising, segmentation, motion estimation, and stereo, to object recognition and image editing. To give researchers some guidance as to which optimization method is best suited for their MRF models, Szeliski et al. [66] conducted a comparative study on 4-connected MRF models. Along with the study, they provided a unifying software framework that facilitated a fair comparison of optimization techniques. The study was well-received in computer vision community and has till date been cited more than 700 times.

Since 2006 when the study was published, the field has made rapid progress. Modern vision problems involve more

complex models, larger datasets and use machine learning techniques to train model parameters.

To summarize, these changes gave rise to challenging energy minimization problems that fundamentally differ from those considered by Szeliski et al. In particular, in [66] the models were restricted to 4-connected grid graphs with unary and pairwise factors only, whereas modern ones include arbitrary structured graphs and higher order potentials.

It is time to revisit the study [66]. We provide a modernized comparison, updating both the problem instances and the inference techniques. Our models are different in the following four aspects:

1. Higher order models, e.g. factors of order up to 300;
2. Models on "regular" graphs with a denser connectivity structure, e.g. 27-pixel neighborhood, or models on "irregular" graphs with spatially non-uniform connectivity structure;
3. Models based on superpixels with smaller number of variables;
4. Image partitioning models without unary terms and unknown number of classes.

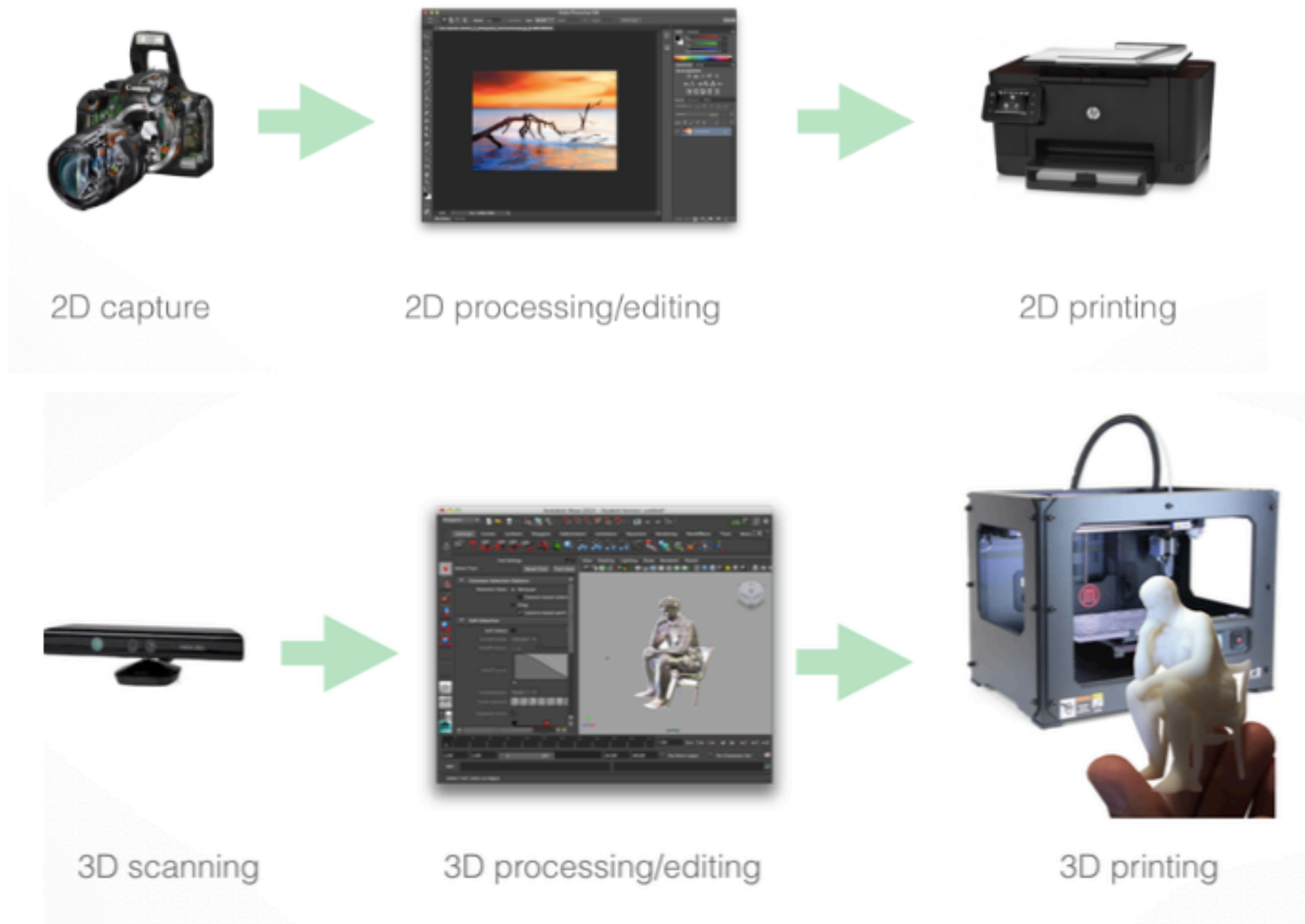
Inference methods have changed since 2006 as well, often as a response to the development of challenging models. The study [66] compared the performance of the state of the art at that time, represented by primal move-making methods, loopy belief propagation, a tree-reweighted message passing, and a set of more traditional local optimization heuristics like iterated conditional modes (ICM).

We augment this set with recent updates of the move-making and local optimization methods, methods addressing higher order models, and polyhedral methods considering the energy minimization as an (Integer) Linear Program.

Contributions We provide a modernized experimental study of energy minimization methods. Our study includes the cases and algorithms studied by [66], but significantly expand it in the scope of both used inference methods and considered models. Both the methods and the considered models are implemented and stored within a single uniform multi-platform software framework, OpenGM 2 [4]. Together with results of

Digitizing objects

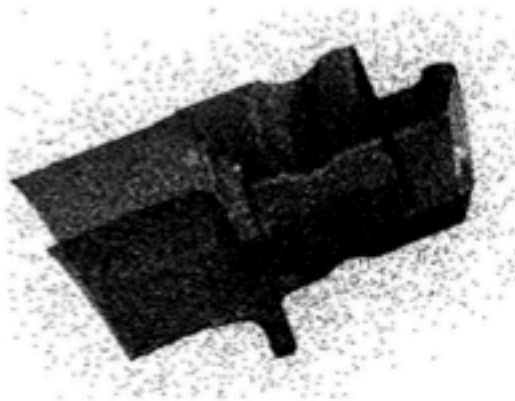
Reproducing real-world objects



Digitizing objects

Point clouds

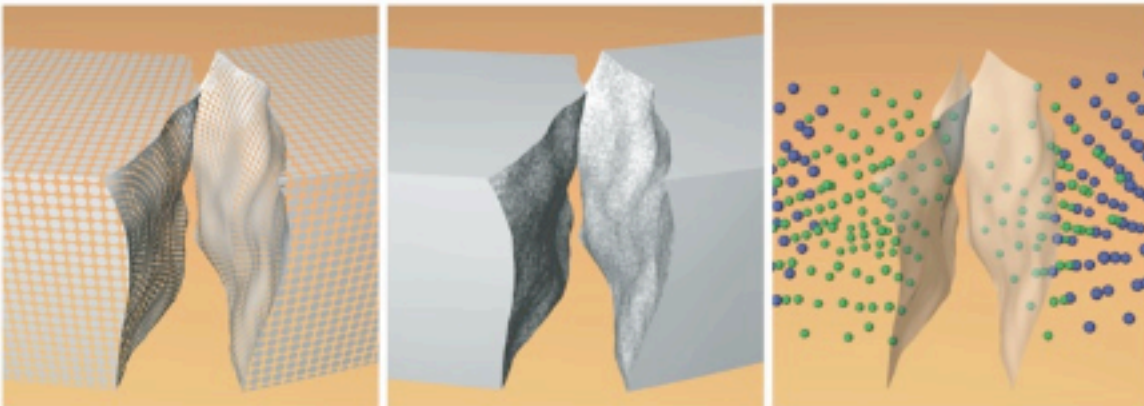
- Simplest representation: **only points**, no connectivity.
- Collection of (x,y,z) coordinates, possibly with normals.
- Points with orientation are called **surfels**.
- Severe limitations:
 - **no** Simplification or subdivision
 - **no** direct smooth rendering
 - **no** topological information
 - Weak approximation power.
 - Noise and outliers



Why point clouds?

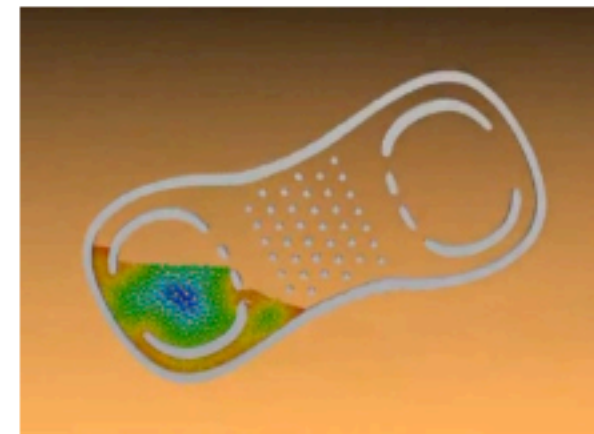
- 1) Typically, that's the only thing that's available
- 2) Locality: sometimes, easier to handle (esp. in hardware).

Fracturing Solids



Meshless Animation of Fracturing Solids
Pauly et al., SIGGRAPH '05

Fluid Simulation

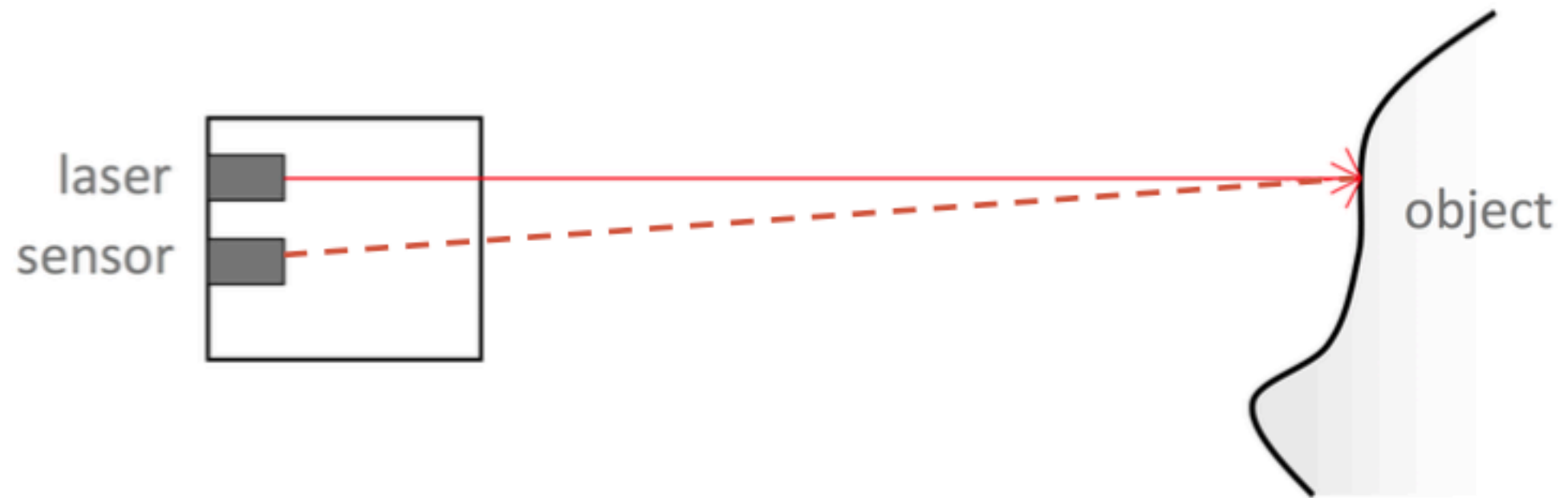


Adaptively sampled particle fluids,
Adams et al. SIGGRAPH '07

Different measurement techniques

- **Range (emission-based) scanners**
 - Time-of-flight laser scanner
 - Phase-based laser scanner
- **Triangulation**
 - Laser line sweep
 - Structured light
- **Stereo / computer vision**
 - Passive stereo
 - Active stereo / space time stereo

Time of flight scanners



1. Emit a short short pulse of laser
2. Capture the reflection.
3. Measure the time it took to come back.
4. Need a very fast clock.
5. Main advantage: can be done over long distances.
6. Used in terrain scanning.

Time of flight scanners



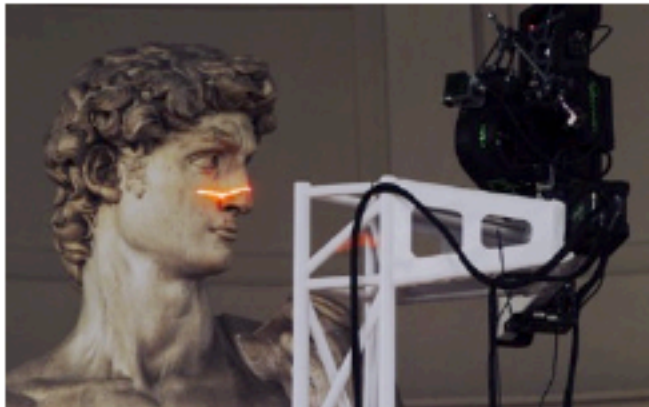
source: Michael Wand

Structured light

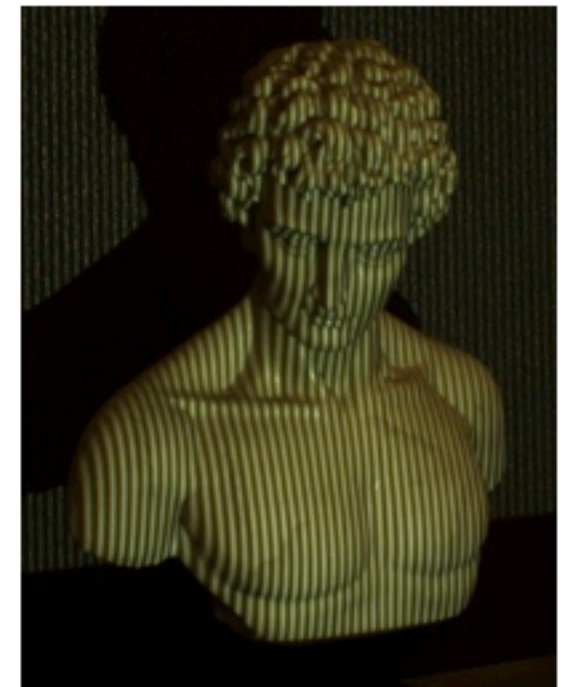
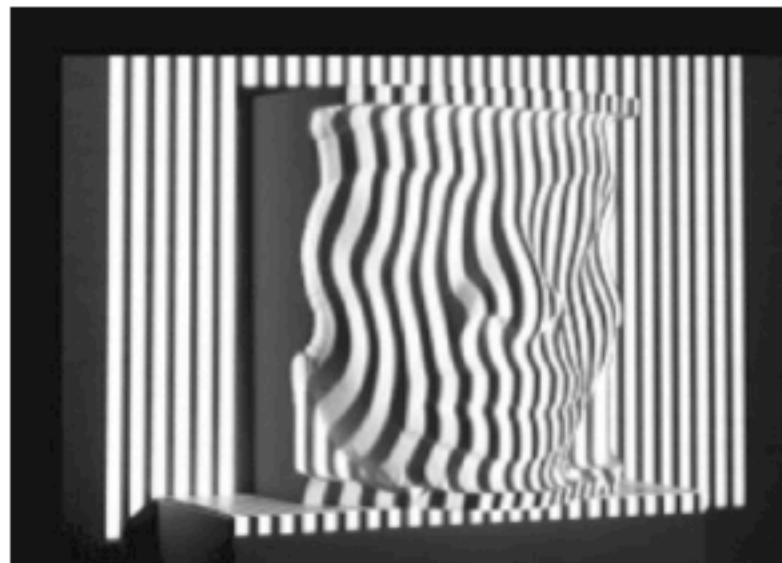
Same general idea as Triangulation based scanner.

Main Idea: Replace laser with projector. Project stripes instead of sheets.

Challenge: Need to identify which (input/output) lines correspond.

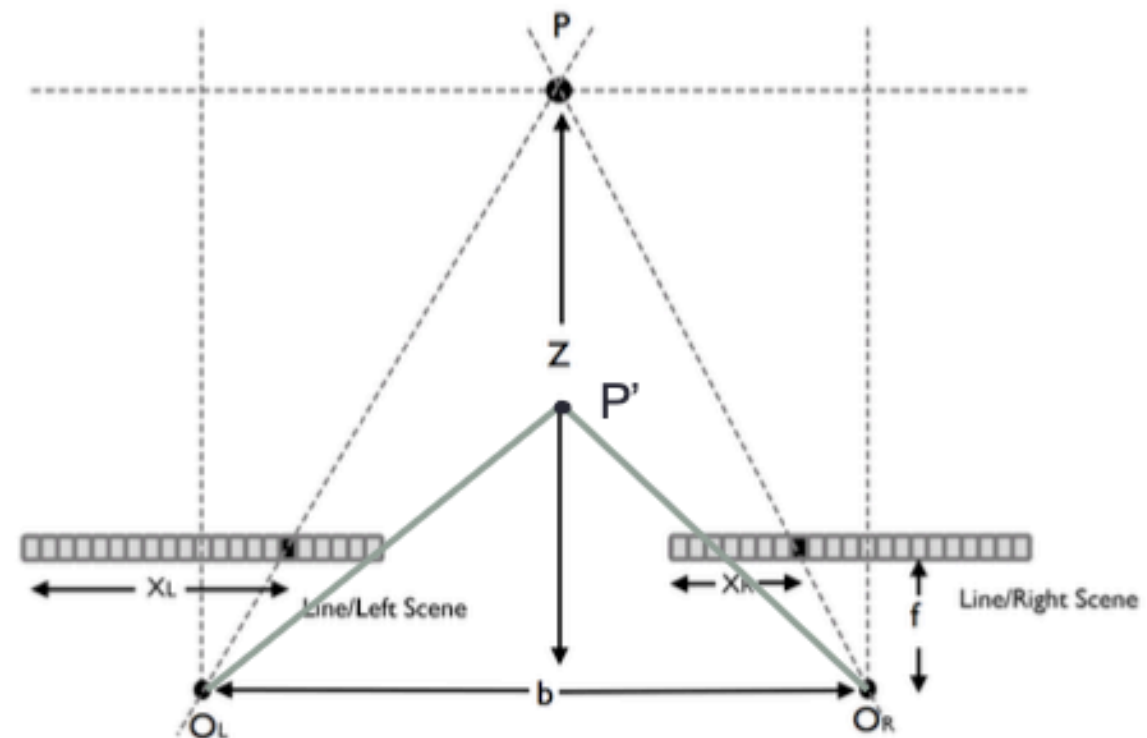
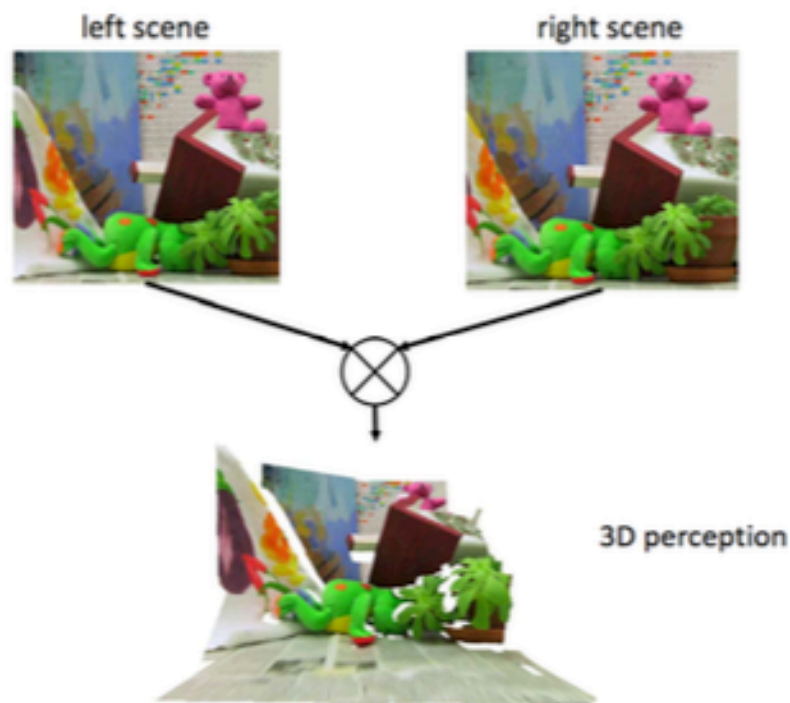


vs.



„Our“ approach in this lecture

Depth from stereo:



Given 2 images, shift in the x-axis is related to the depth.

Main challenge: establishing corresponding points across images: **very difficult.**

Depth-from-blur

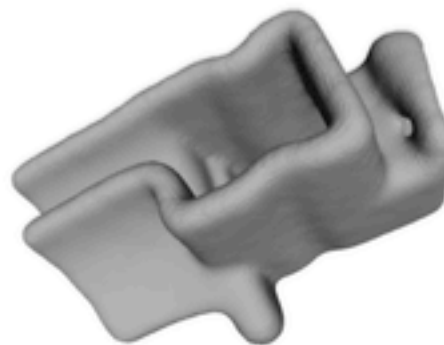
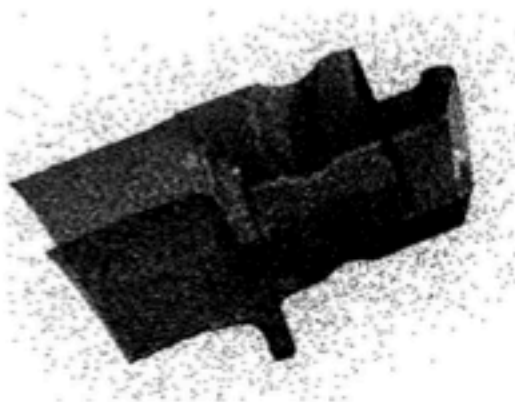


Can approximate depth by detecting how blurry part of the image is for **known focal length.**

Point cloud processing

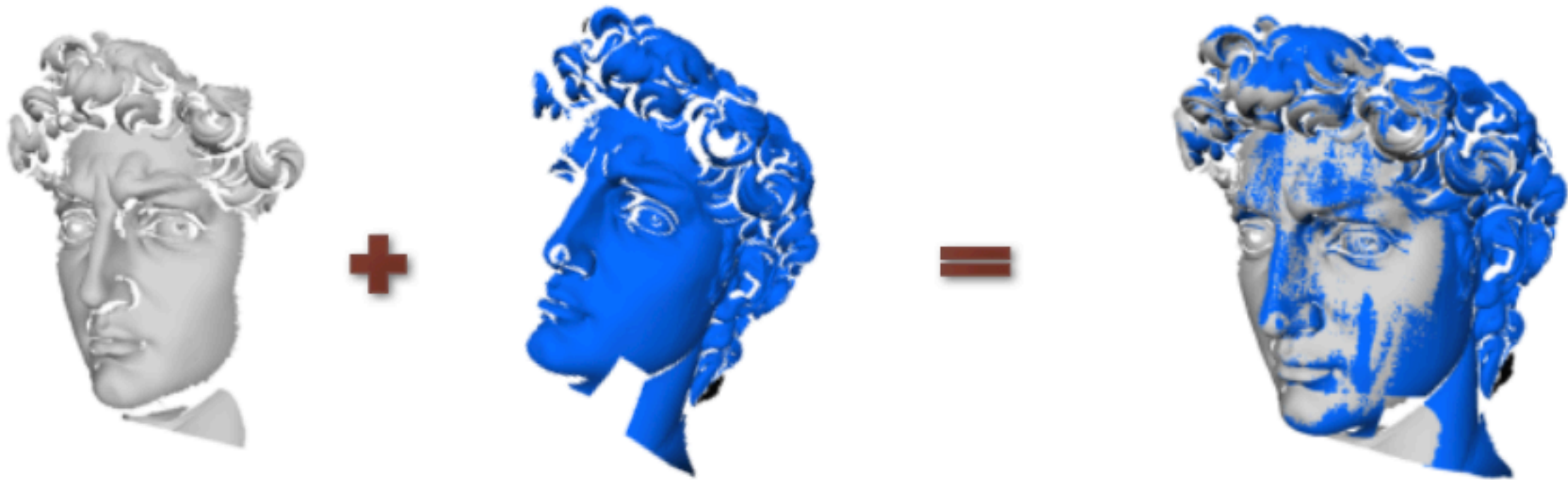
Typically point cloud sampling of a shape is insufficient for most applications. Main stages in processing:

1. Shape scanning (acquisition)
2. **If have multiple scans, align them.**
3. Smoothing – remove local noise.
4. Estimate surface normals.
5. Surface reconstruction
 - Implicit representation (today).
 - Triangle mesh (today).



Point cloud registration

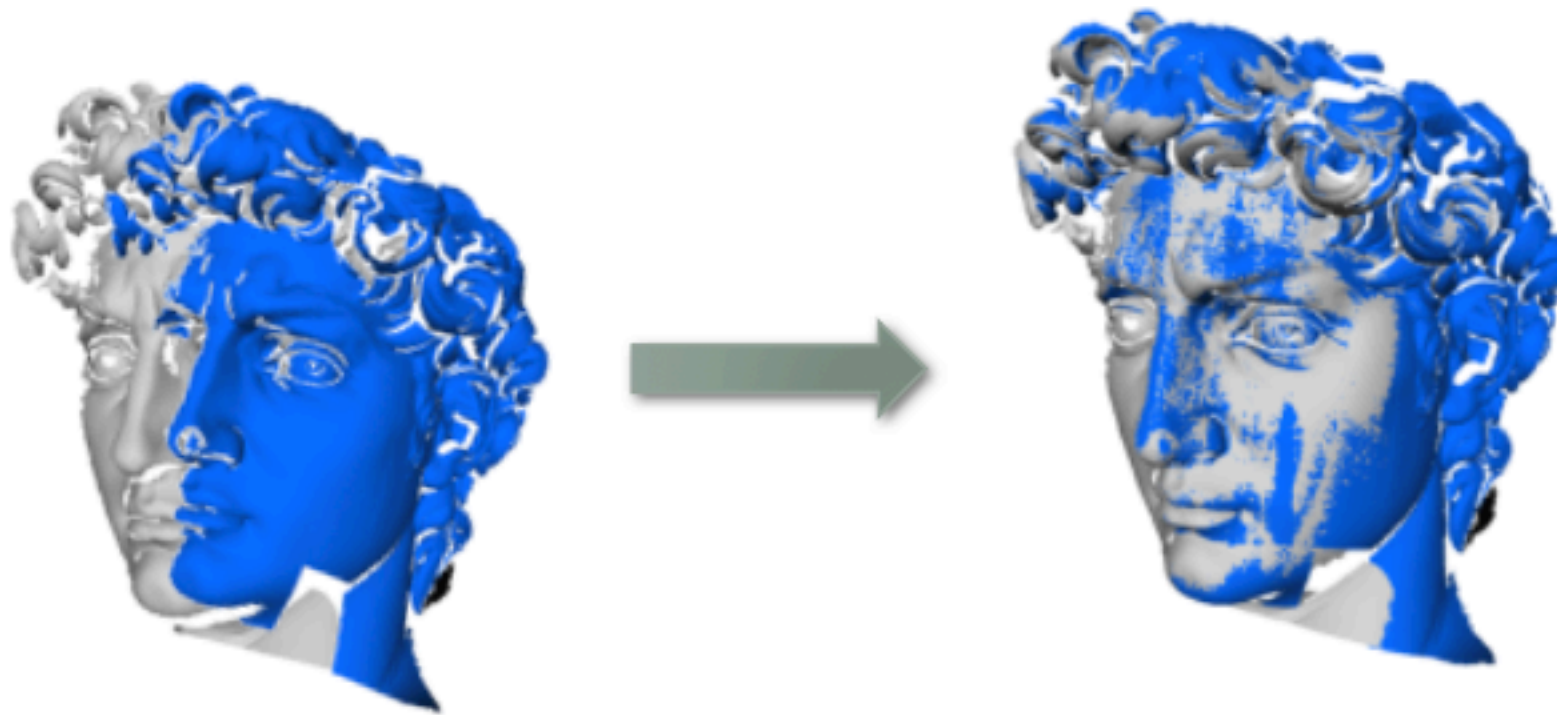
Registration problem



Given (at least) two shapes with partially overlapping geometry, find an alignment between them.

Local alignment

- Simplest instance of the registration problem



Given two shapes that are **approximately aligned** (e.g. by a human) we want to find the optimal transformation.

Applications

Manufacturing:

One shape is a **model** and the other is a **scan** of a product. Finding defects.

Medicine:

Finding correspondences between 3D MRI scans of the same person or different people.

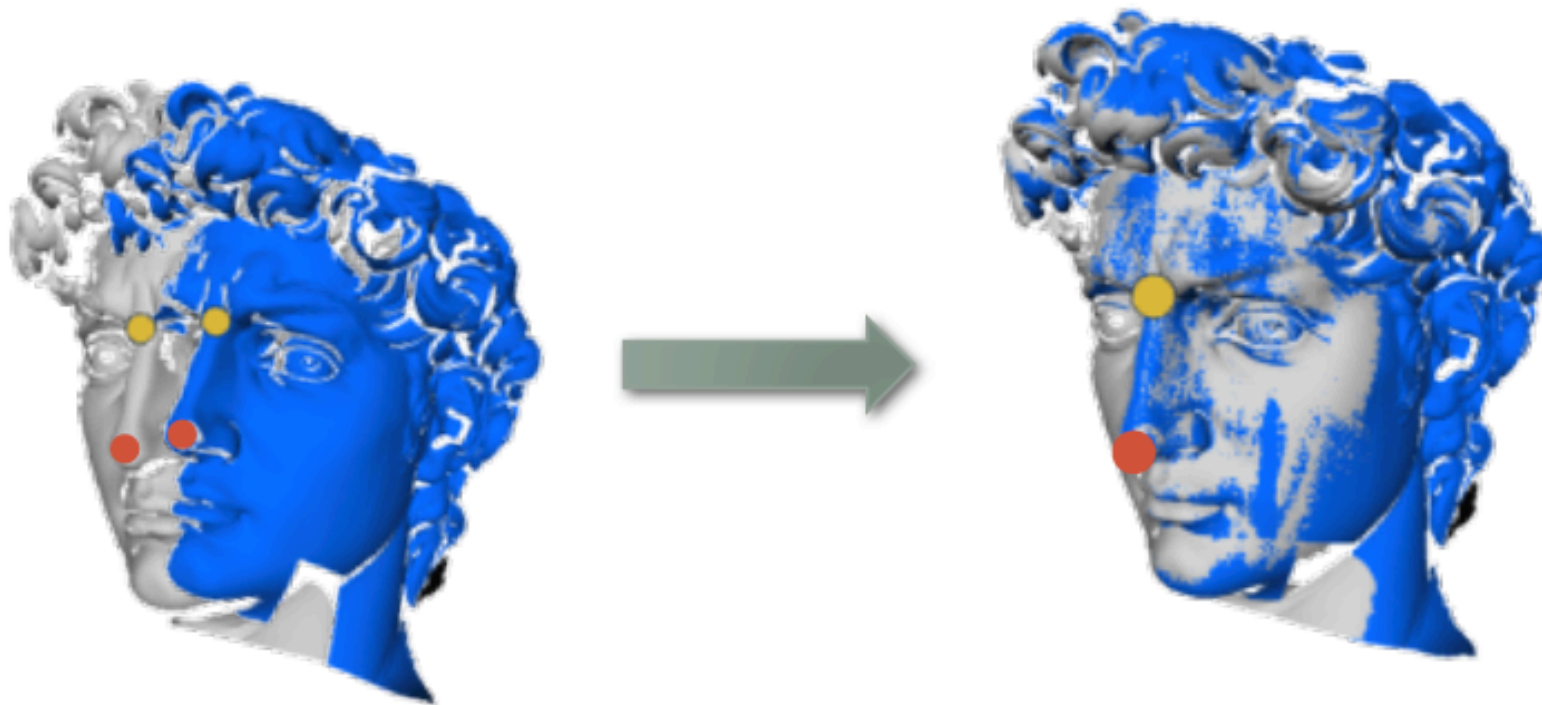
Animation Reconstruction & 3D Video.

Statistical Shape Analysis:

Building models for a collection of shapes.

Local alignment

- What does it mean for an alignment to be **good**?



Intuition: want corresponding points to be close after transformation.

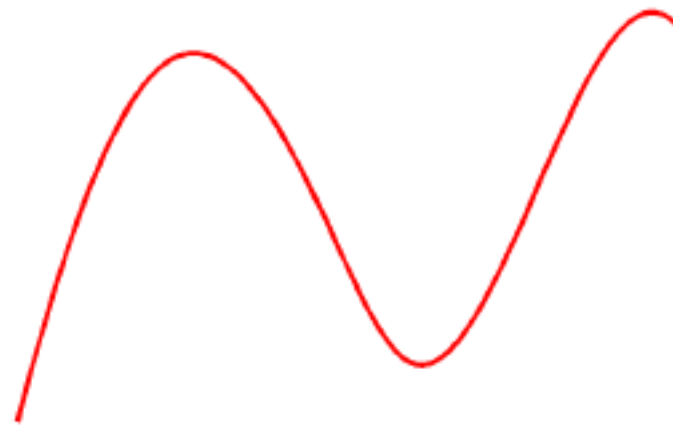
Problems

1. We don't know what points correspond.
2. We don't know the optimal alignment.

**Classical solution:
Iterative closest point (ICP)**

Iterative closest point (ICP)

- Approach: iterate between finding correspondences and finding the transformation:



Given a pair of shapes, X and Y , iterate:

1. For each $x_i \in X$ find **nearest** neighbor $y_i \in Y$.
2. Find deformation $\mathbf{R}, t$ minimizing:

$$\sum_{i=1}^N \|\mathbf{R}x_i + t - y_i\|_2^2$$

Iterative Closest Point (ICP)

- Requires two main computations:
 1. Computing nearest neighbors.
 2. Computing the optimal transformation



Iterative Closest Point (ICP)

$$y_i = \arg \min_{y \in Y} \|y - x_i\|$$

- How to find closest points **efficiently**?

- Straightforward complexity: $\mathcal{O}(MN)$

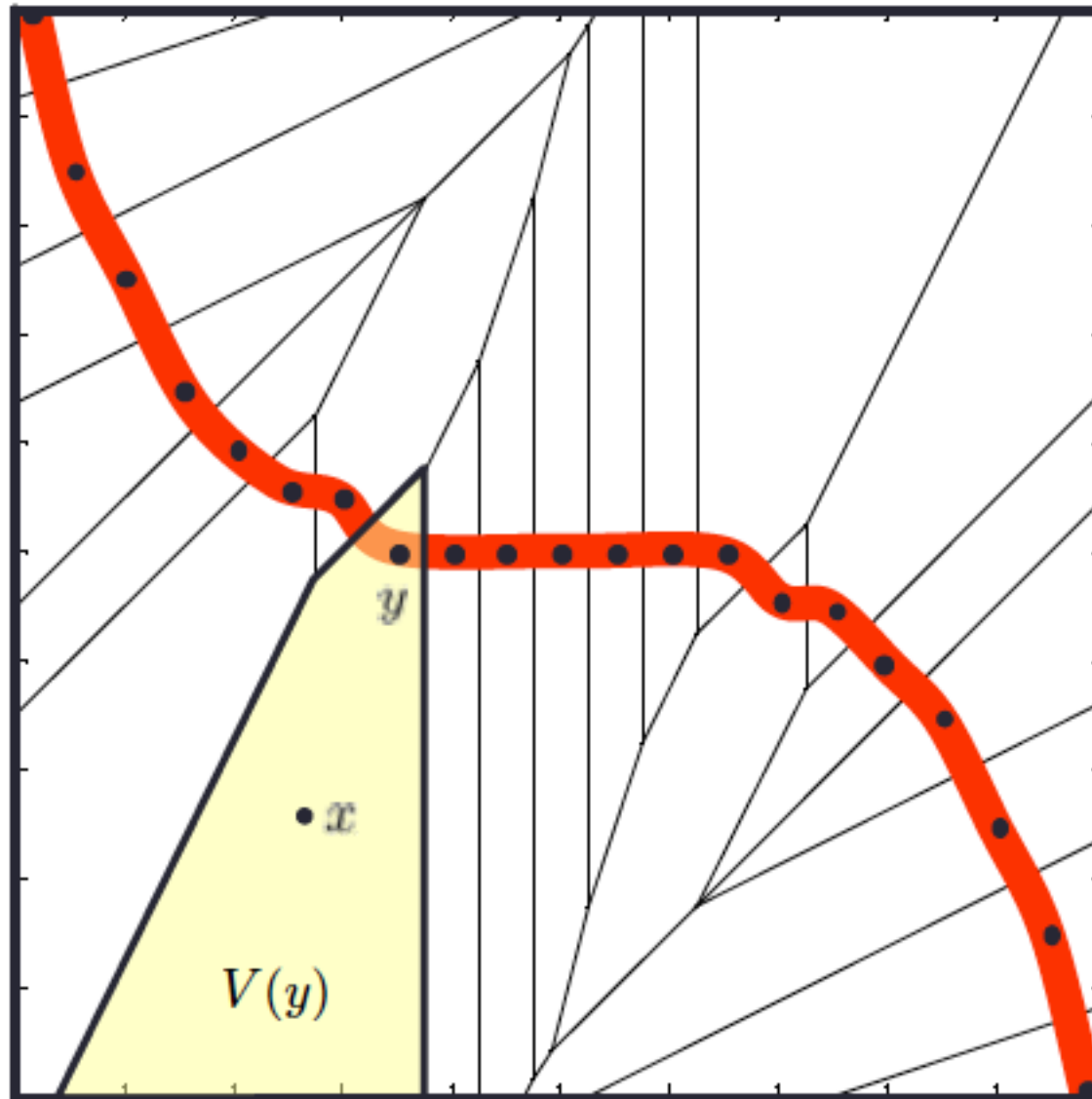
M number of points on X , N number of points on Y .

- Y divides the space into **Voronoi cells**

$$V(y \in Y) = \{z \in \mathbb{R}^3 : \|y - z\| < \|y' - z\| \ \forall \ y' \in Y \neq y\}$$

- Given a query point y , **determine to which cell it belongs.**

Voronoi cells

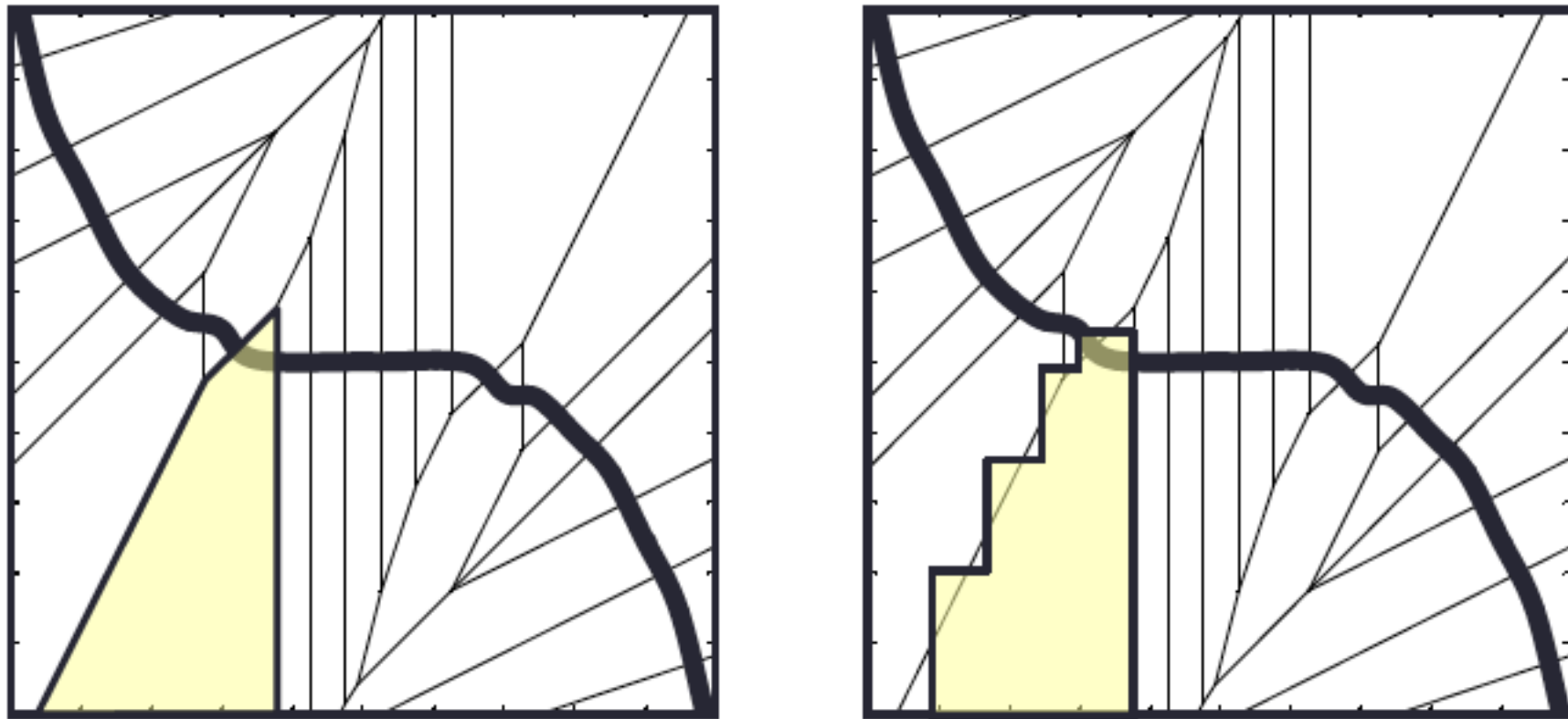


$$V(y \in Y) = \{z \in \mathbb{R}^3 : \|y - z\| < \|y' - z\| \ \forall \ y' \in Y \neq y\}$$

Source:
M. Bronstein

Voronoi cells

Approximate nearest neighbors



M. Bronstein

- To reduce search complexity, **approximate** Voronoi cells.
- Use **binary space partition trees** (e.g. **kd-trees** or **octrees**).
- Approximate nearest neighbor search complexity: $\mathcal{O}(N \log M)$.

Optimal transformation in ICP

Problem Formulation:

1. Given two sets points: $\{x_i\}, \{y_i\}, i = 1..n$ in $\mathbb{R}^3$. Find the rigid transform:

$\mathbf{R}, t$ that minimizes:

$$\sum_{i=1}^N \|\mathbf{R}x_i + t - y_i\|_2^2$$

2. Closed form solution with rotation matrices:

1. Construct: $C = \sum_{i=1}^N (y_i - \mu^Y)(x_i - \mu^X)^T$, where $\mu^X = \frac{1}{N} \sum_i x_i$,

2. Compute the SVD of C: $C = U\Sigma V^T$ $\mu^Y = \frac{1}{N} \sum_i y_i$

1. If $\det(UV^T) = 1, R_{\text{opt}} = UV^T$

2. Else $R_{\text{opt}} = U\tilde{\Sigma}V^T, \tilde{\Sigma} = \text{diag}(1, 1, \dots, -1)$

3. Set $t_{\text{opt}} = \mu^Y - R_{\text{opt}}\mu^X$

Note that C is a 3x3 matrix. SVD is very fast.

Arun et al., Least-Squares Fitting
of Two 3-D Point Sets

Iterative Closest Point (ICP)

Given a pair of shapes, X and Y , iterate:

1. For each $x_i \in X$ find **nearest** neighbor $y_i \in Y$.
2. Find deformation $\mathbf{R}, t$ minimizing: $\sum_{i=1}^N \|\mathbf{R}x_i + t - y_i\|_2^2$

Convergence:

- at each iteration $\sum_{i=1}^N d^2(x_i, Y)$ decreases.
- Converges to local minimum
- Good initial guess: global minimum.

[Besl&McKay92]

More stable version

Deal with irregular sampling

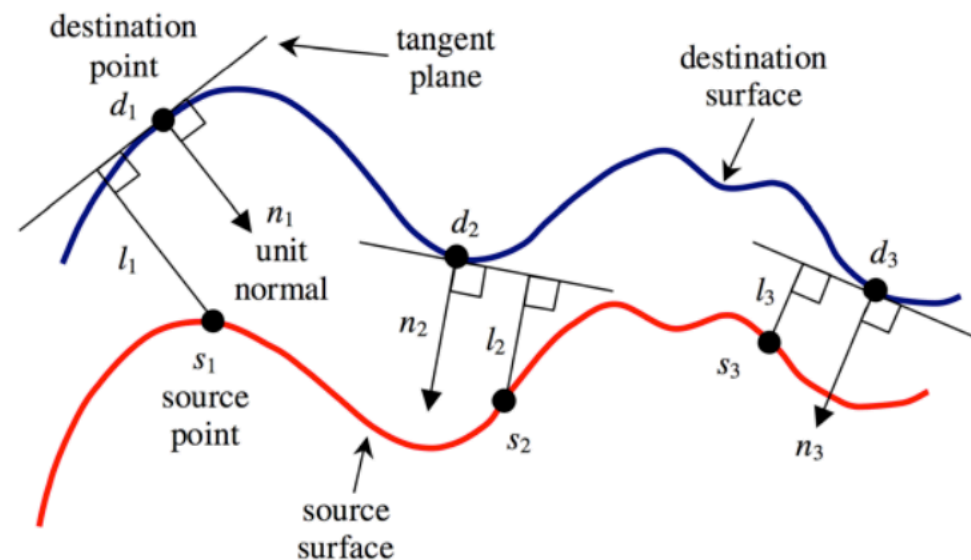
Given a pair of shapes, X and Y , iterate:

1. For each $x_i \in X$ find **nearest** neighbor $y_i \in Y$.
2. Find deformation $\mathbf{R}, t$ minimizing:

$$\sum_{i=1}^N d(\mathbf{R}x_i + t, P(y_i))^2 = \sum_{i=1}^N ((\mathbf{R}x_i + t - y_i)^T \mathbf{n}_{y_i})^2$$

Solution:

Minimize distance to the tangent plane



Kok-Lim Low, '04

From point sets to surface representations

Main goal

Construct a polygonal (e.g. triangle mesh) representation of the point cloud.



PCD

Reconstruction
algorithm



curve/ surface

Main problem

Data is **unstructured**. E.g. in 2D the points are not **ordered**.
Inherently **ill-posed** (aka difficult) problem.



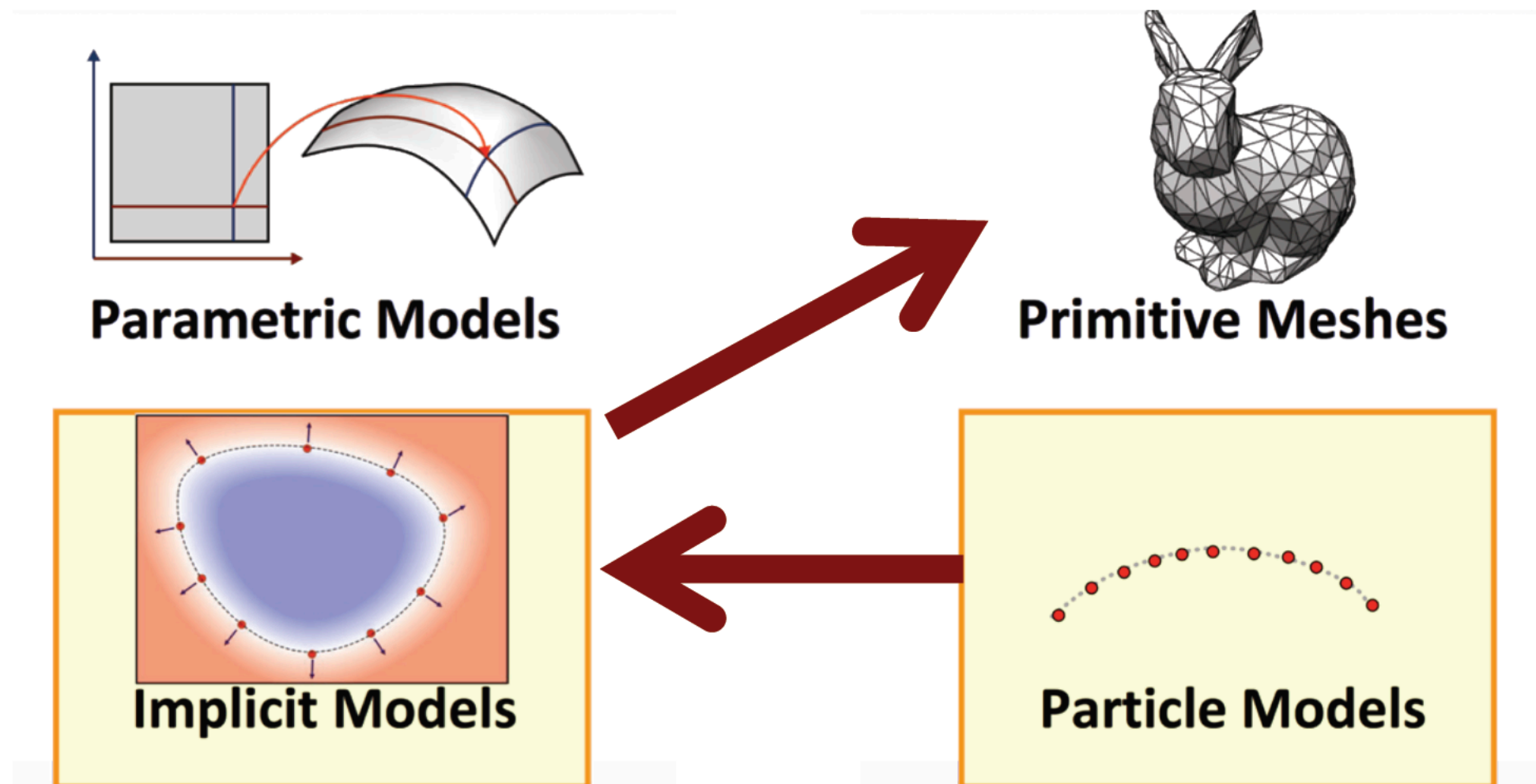
PCD

Reconstruction
algorithm



curve/ surface

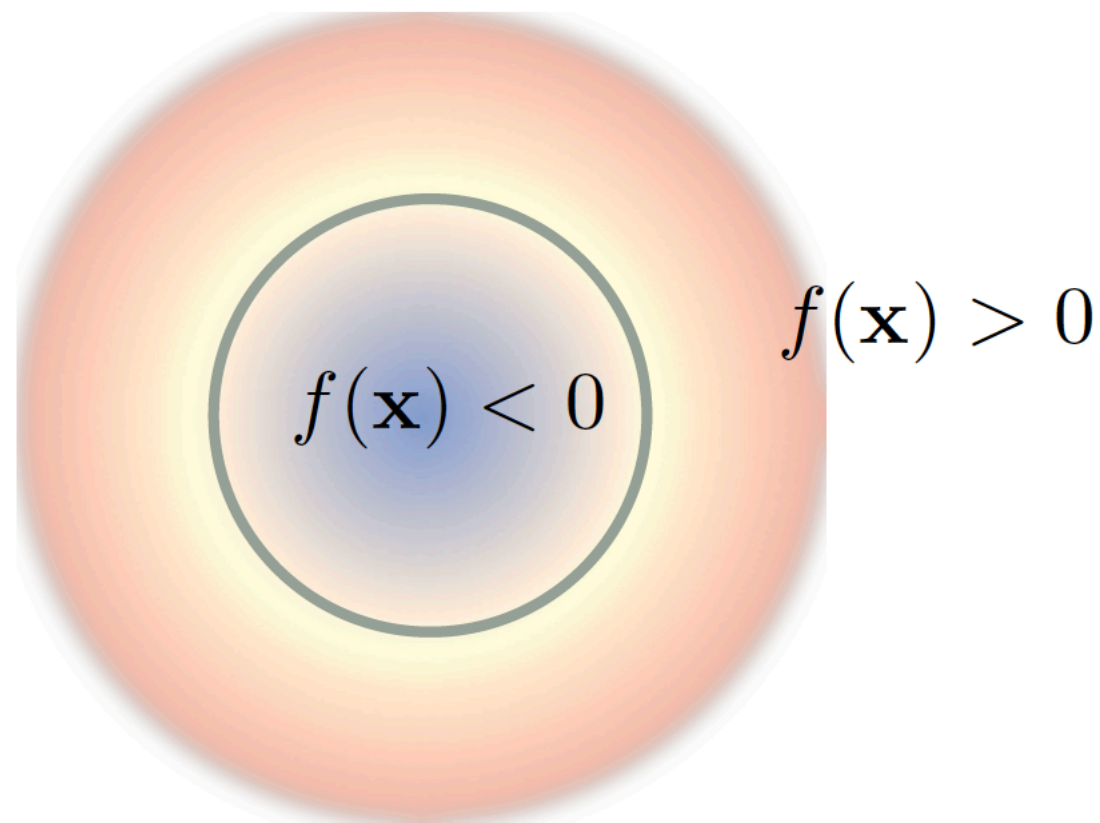
Possible solution: Estimate meshes via implicit models



Implicit surfaces

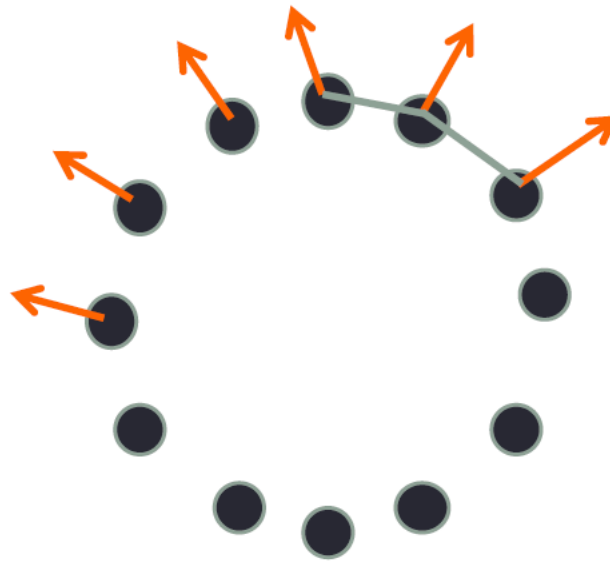
Given a function $f(\mathbf{x})$, the surface is defined as:

$$\{\mathbf{x}, \text{s.t. } f(\mathbf{x}) = 0\}$$



$$f(x, y) = x^2 + y^2 - r^2$$

Implicit surfaces



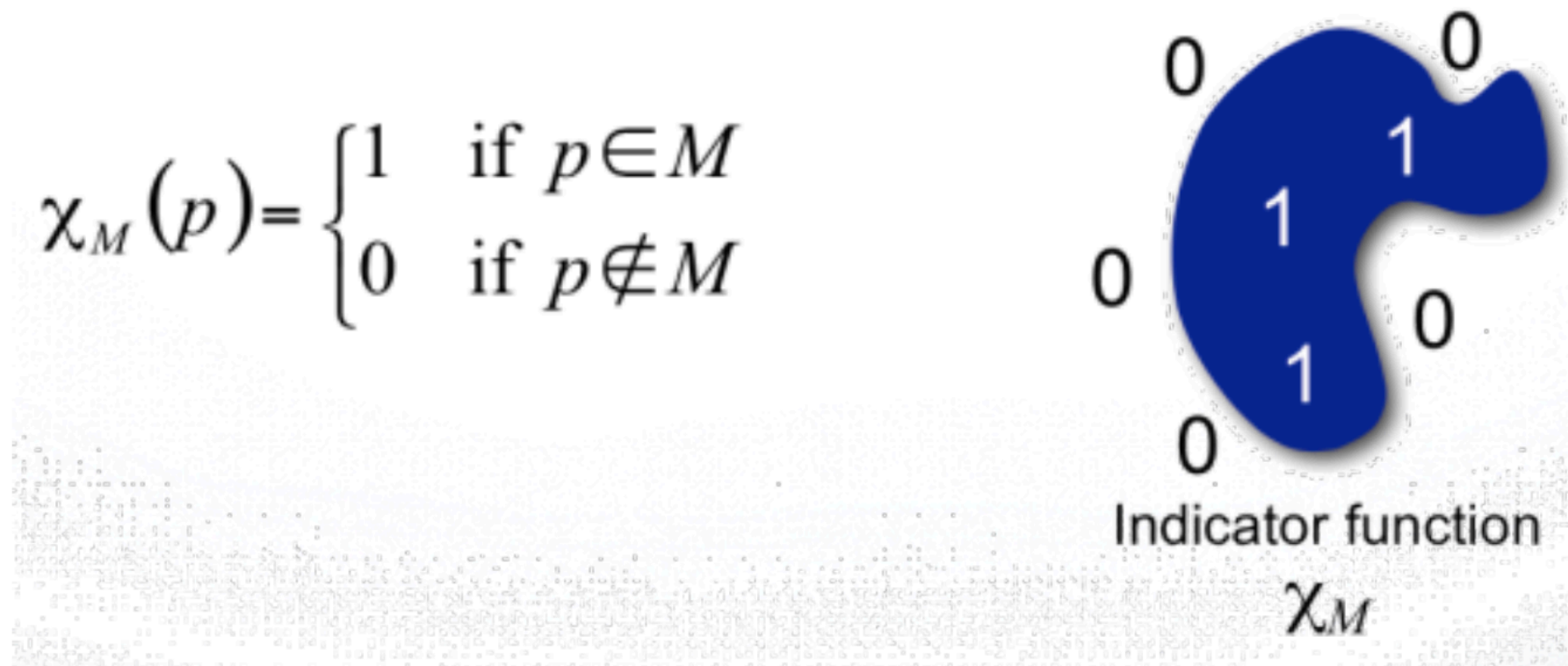
Simplest method:

1. Given a point x in space, find nearest point p in PCD.
2. Set $f(x) = (x - p)^T \mathbf{n}_p$ – signed distance to the tangent plane.
3. Note: need consistently oriented normals. In general, difficult problem, but can try to locally connect points and fix orientations.

Hugues Hoppe: Surface reconstruction from unorganized points

Poisson surface reconstruction

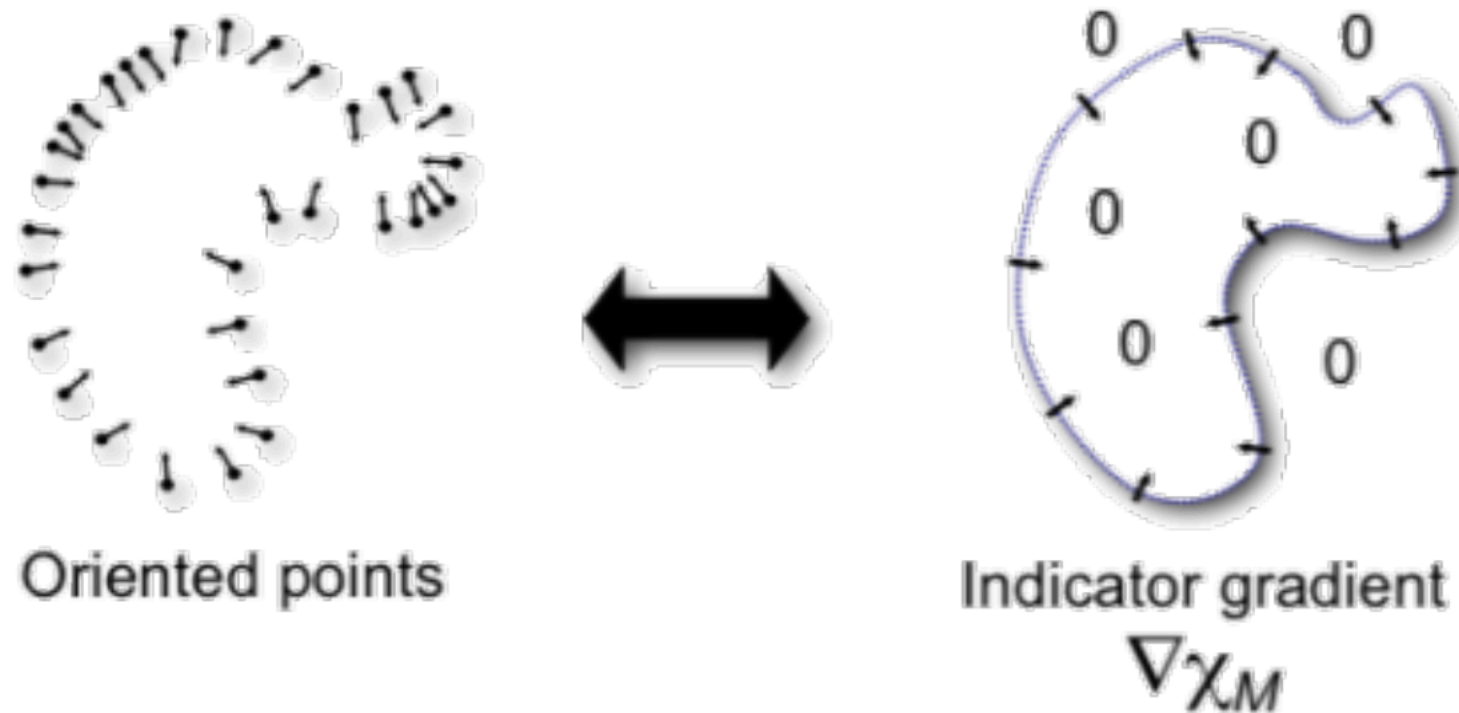
Main Goal: construct an indicator function of a surface



Poisson surface reconstruction, Kazhdan, Bolitho, Hoppe, SGP '06

Poisson surface reconstruction

Solving for the indicator function:



$$\min_{\chi} \|\nabla \chi - V\|$$

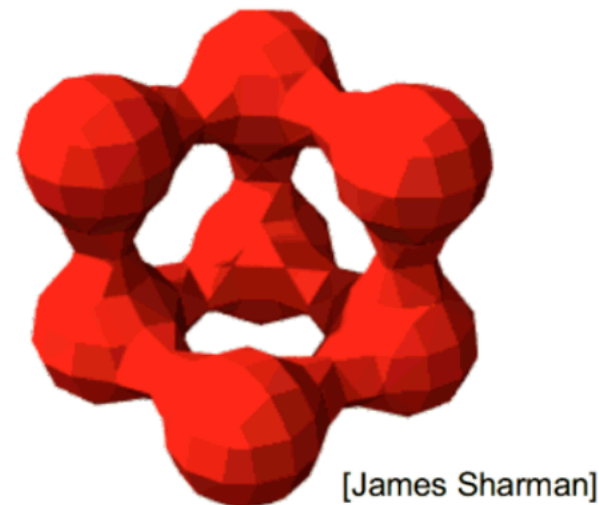
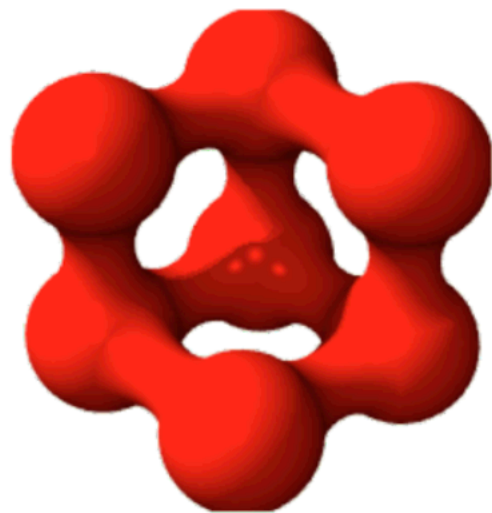
Marching cubes: From implicit representation to mesh

Marching cubes

Converting from implicit to explicit representations.

Goal: Given an implicit representation: $\{\mathbf{x}, \text{s.t. } f(\mathbf{x}) = 0\}$

Create a triangle mesh that approximates the surface.



[James Sharman]

Lorensen and Cline, SIGGRAPH '87

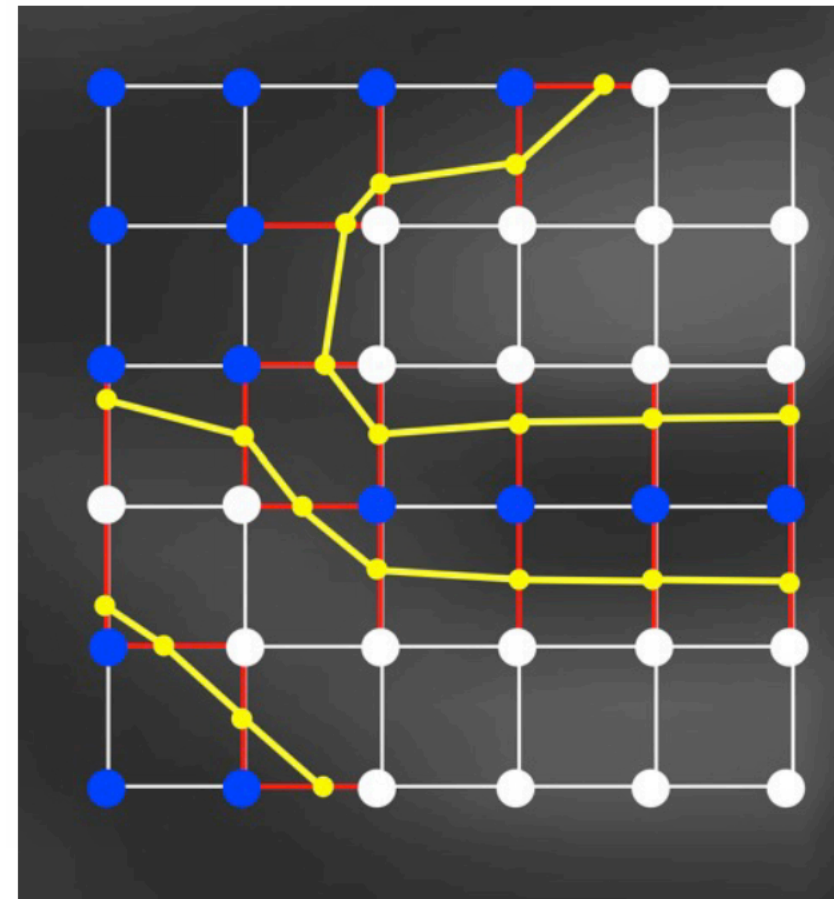
One of the most cited computer graphics papers of all time.

2D: Marching squares

Given a function: $f(x)$

- $f(\mathbf{x}) < 0$ inside
- $f(\mathbf{x}) > 0$ outside

1. Discretize space.
2. Evaluate $f(x)$ on a grid.
3. Classify grid points (+/-)
4. Classify grid edges
5. Compute Intersections
6. Connect Intersections



2D: Marching squares

Computing the intersections:

- Edges with a sign switch contain intersections.

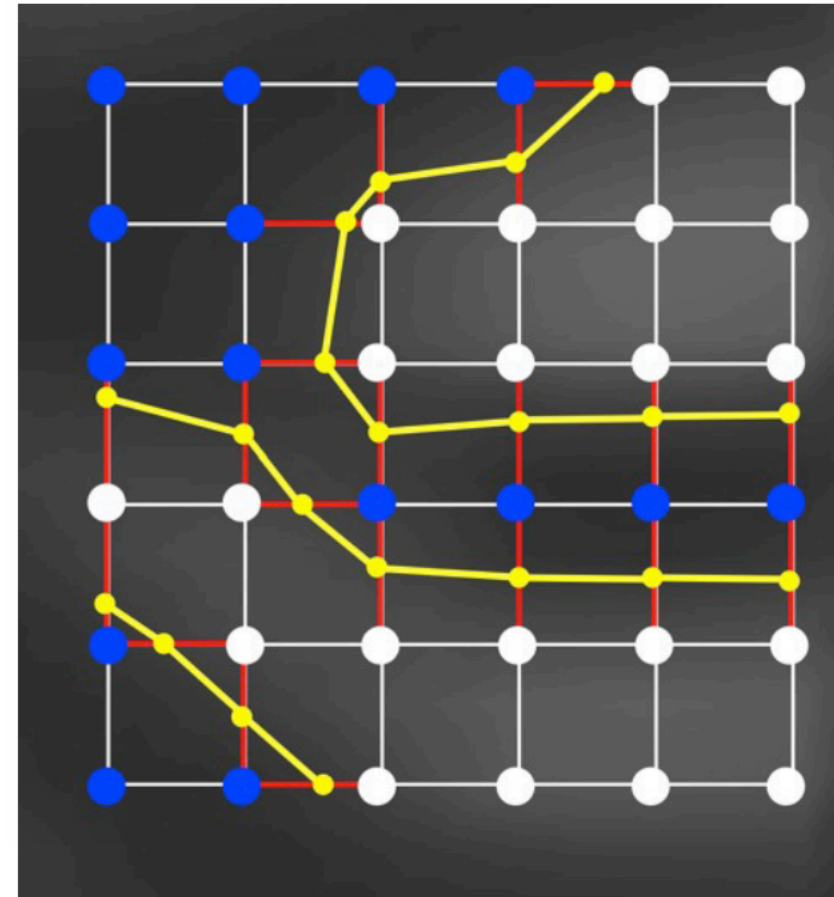
$$f(x_1) < 0, f(x_2) > 0 \Rightarrow$$

$$f(x_1 + t(x_2 - x_1)) = 0$$

for some $0 \leq t \leq 1$

- Simplest way to compute t : assume f is linear between x_1 and x_2 :

$$t = \frac{f(x_1)}{f(x_2) - f(x_1)}$$



2D: Marching squares

Computing the intersections:

- Edges with a sign switch contain intersections.

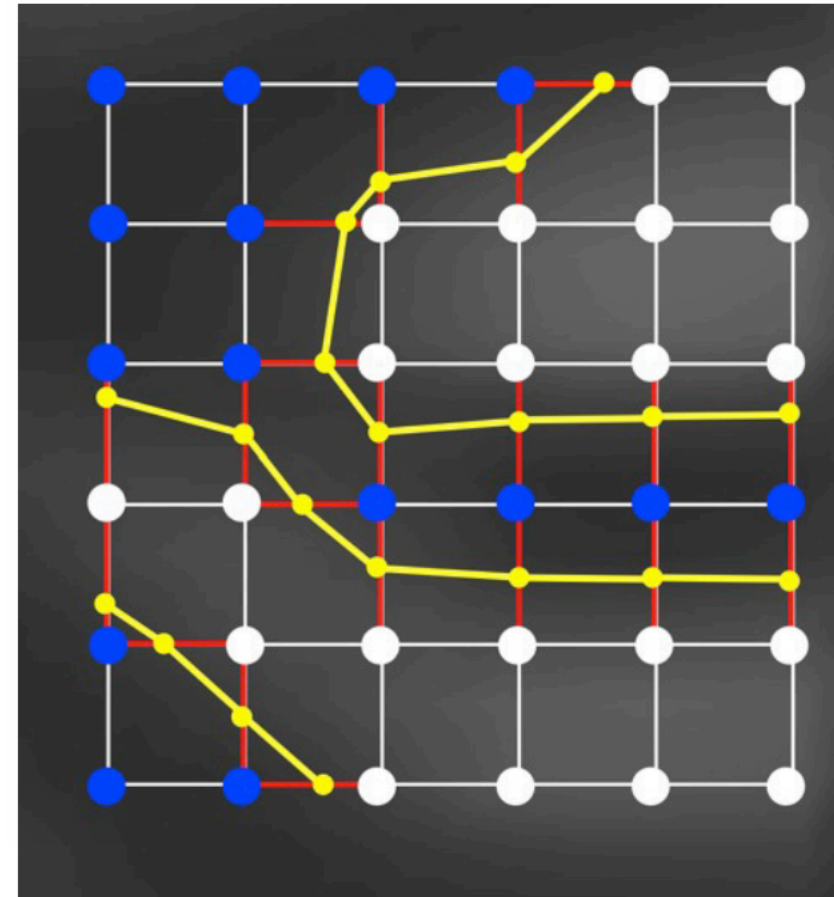
$$f(x_1) < 0, f(x_2) > 0 \Rightarrow$$

$$f(x_1 + t(x_2 - x_1)) = 0$$

for some $0 \leq t \leq 1$

- Simplest way to compute t : assume f is linear between x_1 and x_2 :

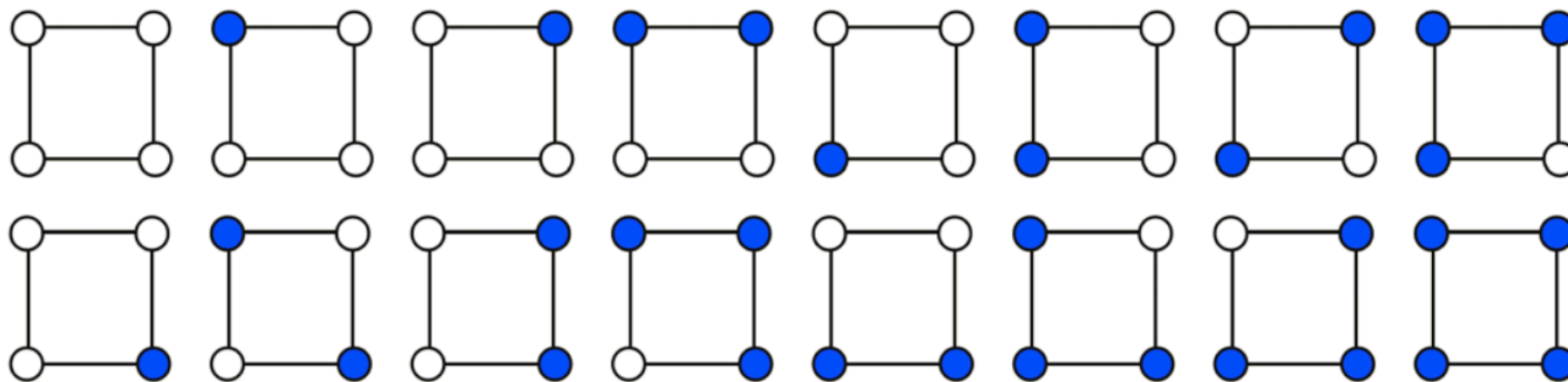
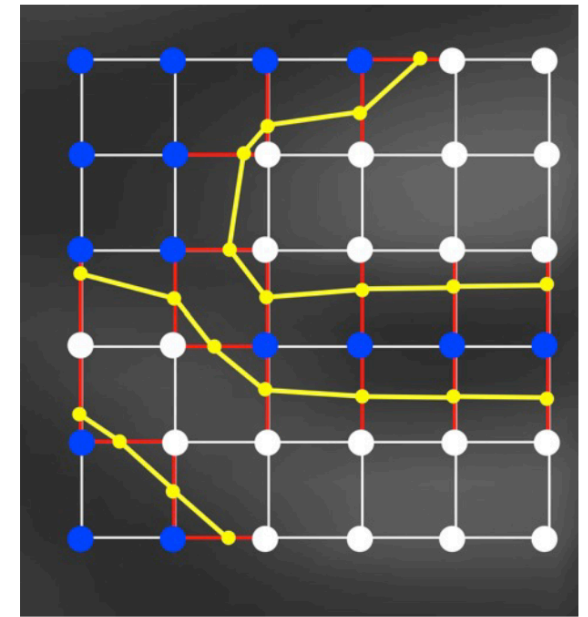
$$t = \frac{f(x_1)}{f(x_2) - f(x_1)}$$



2D: Marching squares

Connecting the intersections:

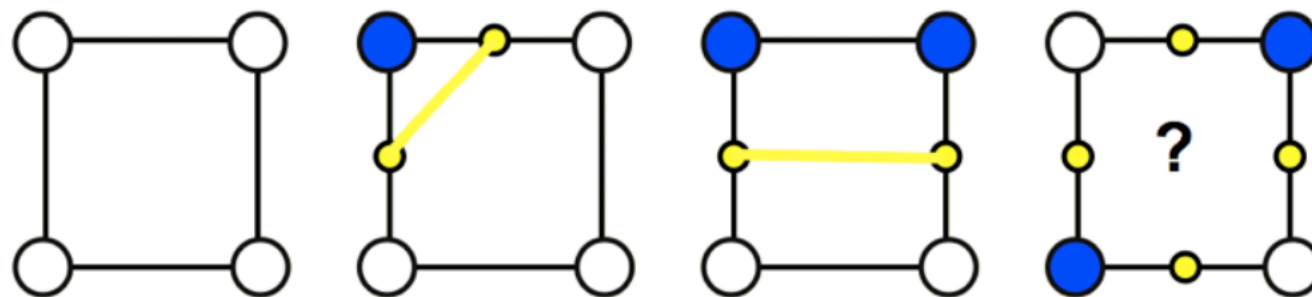
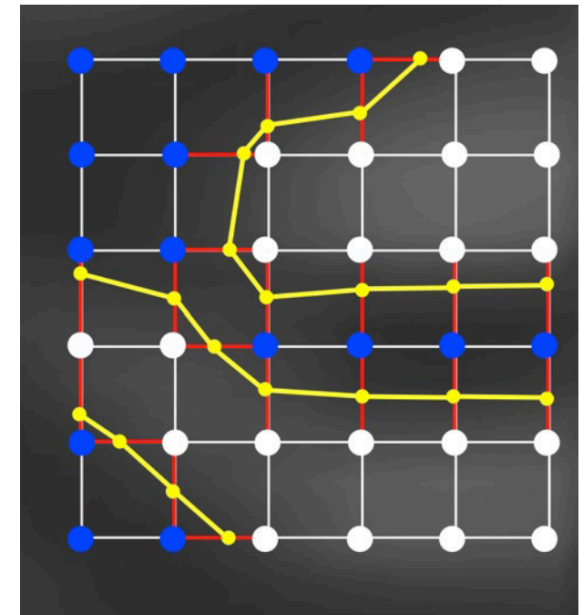
- Grand principle: treat each cell separately!
- Enumerate all possible inside/outside combinations.
- Group those leading to the same intersections



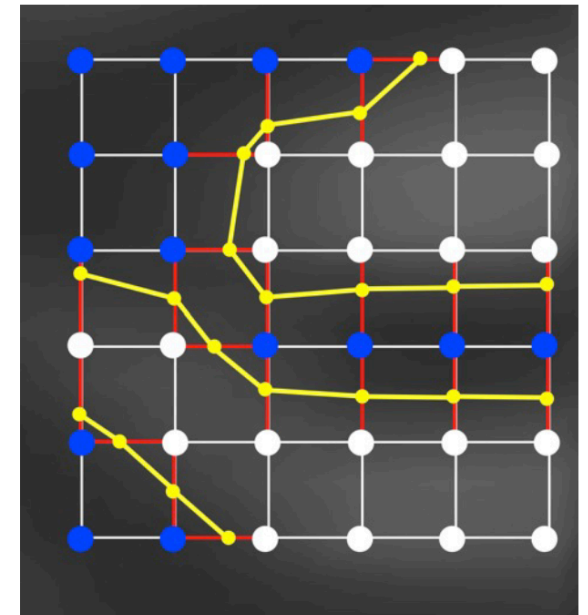
2D: Marching squares

Connecting the intersections:

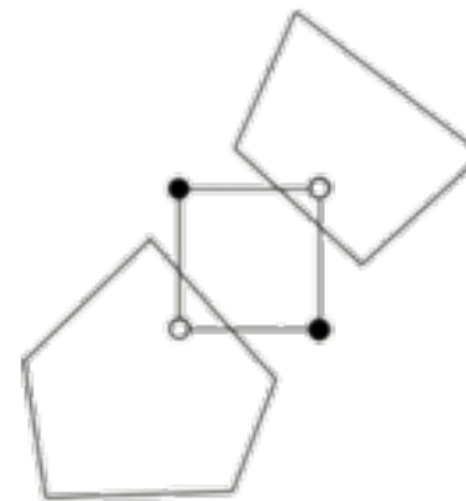
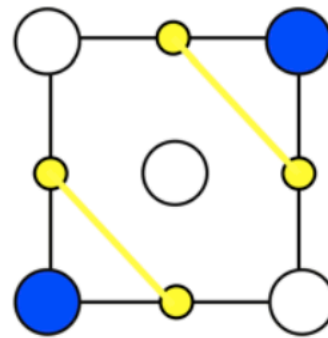
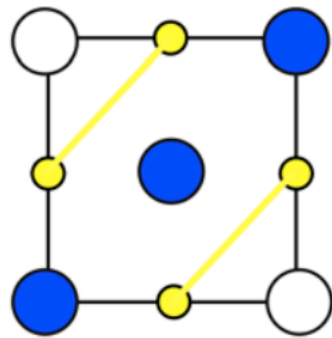
- Grand principle: treat each cell separately!
- Enumerate all possible inside/outside combinations.
- Group those leading to the same intersections.
- Group equivalent after rotation.
- Connect intersections



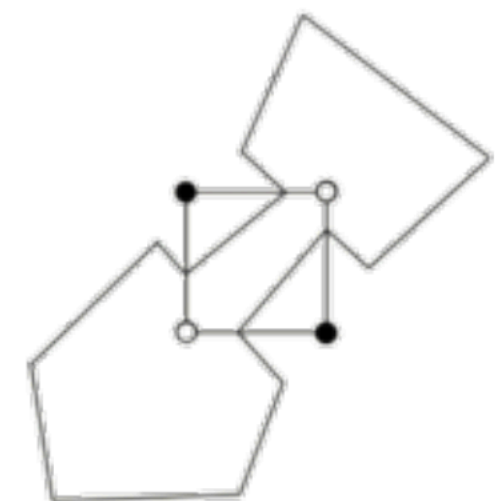
2D: Marching squares



Ambiguous cases:



Break contour



Join contour

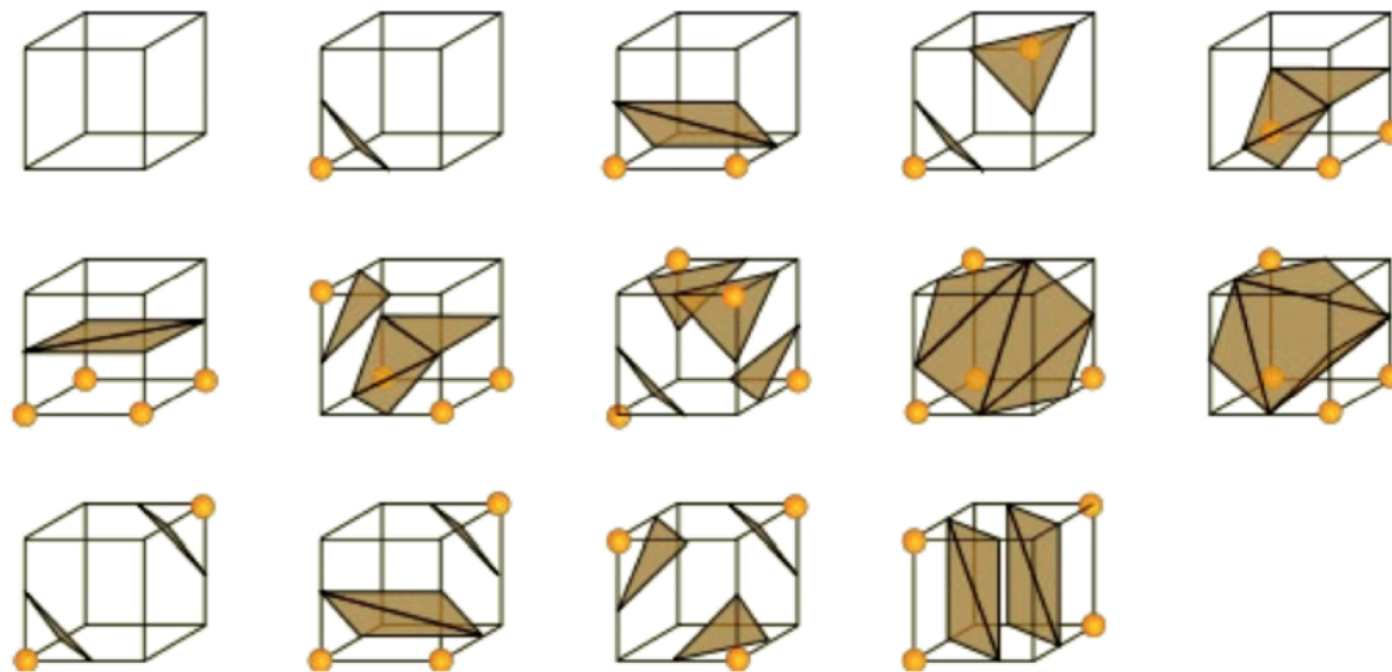
2 options:

- 1) Can resolve ambiguity by subsampling inside the cell.
- 2) If subsampling is impossible, pick one of the two possibilities.

3D: Marching cubes

Same basic machinery applies to 3d.
cells become **cubes** (voxels)
lines become **triangles**

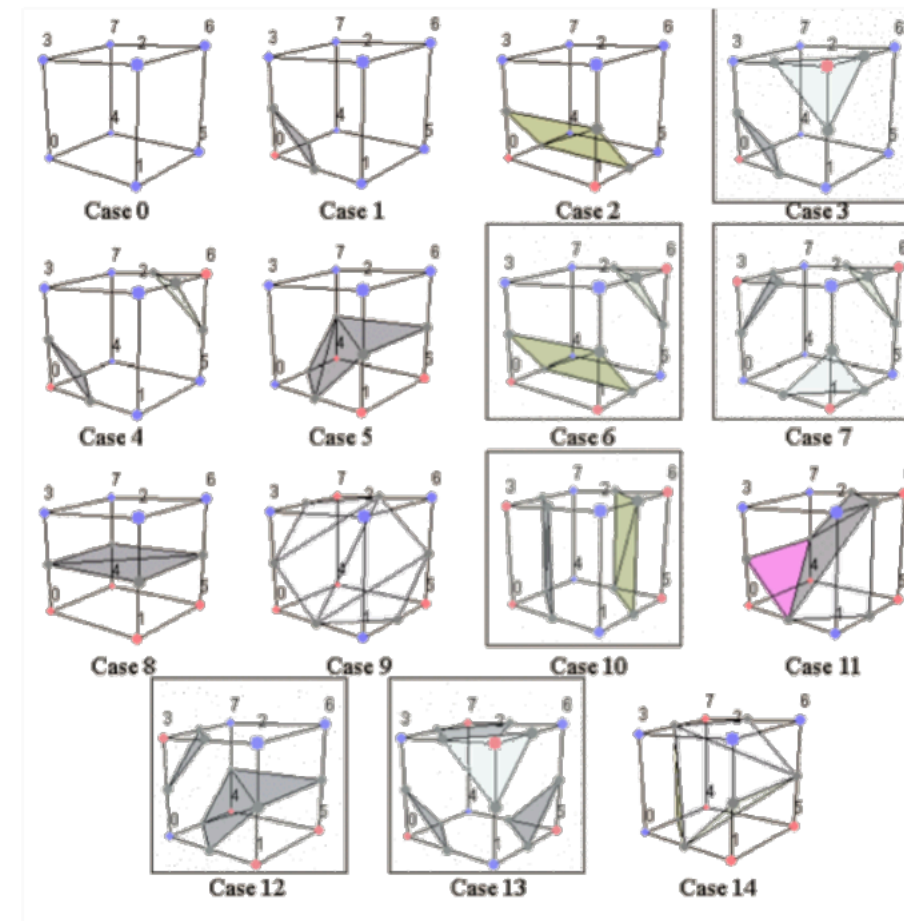
- 256 different cases
- 14 after symmetries



3D: Marching cubes

Same basic machinery applies to 3d.
cells become **cubes** (voxels)
lines become **triangles**

- 256 different cases
- 14 after symmetries
- 6 ambiguous cases (in boxes)



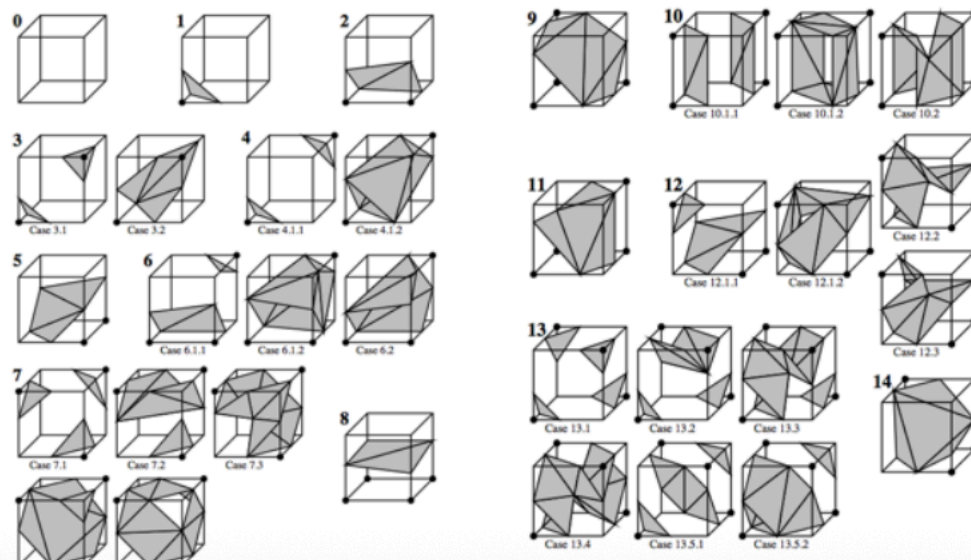
3D: Marching cubes

Same basic machinery applies to 3d.

cells become **cubes** (voxels)

lines become **triangles**

- 256 different cases
- 14 after symmetries
- 6 ambiguous cases (in boxes)
- Inconsistent triangulations can lead to holes and wrong topology.
- More subsampling rules – leads to 33 unique cases.



Chernyaev, Marching Cubes 33,'9

3D: Marching cubes

Main Strengths:

- Very multi-purpose.
- Extremely fast and parallelizable.
- Relatively simple to implement.
- Virtually parameter-free

Main Weaknesses:

- Can create badly shaped (skinny) triangles.
- Basic versions do not provide topological guarantees.
- Many special cases (implemented as big lookup tables).
- No sharp features.